Lightweight CNN Architecture for Bangla Handwritten Digit Recognition

Md. Johirul Islam

Student ID: 21701018

Session: 2020-2021

Supervisor: Dr. Iqbal Ahmed

Professor

Department of Computer Science and Engineering

UNIVERSITY OF CHITTAGONG

CSECU August 2026

Report Code:

University of Chittagong

Department of Computer Science and Engineering

8th Semester B.Sc. Engineering Examination 2024

Course No.: CSE 800

Title: Lightweight CNN Architecture for Bangla Handwritten Digit Recognition

Report Code:

University of Chittagong

Department of Computer Science and Engineering

8th Semester B.Sc. Engineering Examination 2024

Course No.: CSE 800

Student Name: Md. Johirul Islam
Student ID: 21701018
Session: 2020-2021
Hall: Shaheed Abdur Rab Hall

Signature of Student:

Submission Date: August 2026

Supervisor Approval Page

Title of the Thesis: Lightweight CNN Architecture for Bangla Handwritten Digit Recognition
Document Type: Bachelor of Science in Engineering Thesis
Degree Program: Bachelor of Science in Engineering in Computer Science and Engineering
Institution: Department of Computer Science and Engineering, University of Chittagong
Date of Submission: August 2026

Table 1 Evaluation Criteria (to be filled by supervisor(s))

Evaluation Criteria	**Select an Option**		
Maintained Regular Communication ?	◯ *Yes*	◯ *No*	◯ *Partly*
Maintained Professionalism ?	◯ *Yes*	◯ *No*	◯ *Partly*
Addressed the given comments ?	◯ *Yes*	◯ *No*	◯ *Partly*
Checked by Plagiarism and AI content checker ?	◯ *Yes*	◯ *No*	◯ *Partly*
Report	◯ *Satisfactory*	◯ *Not Satisfactory*	◯ *Partly*
Approved	◯ *Yes*	◯ *No*	

This B.Sc. Engg. Thesis has been reviewed and (NOT) approved by _____

_____

considering the evaluation criteria outlined in Table 1.

_____

Signature

Dr. Iqbal Ahmed
Professor
Department of Computer Science and Engineering
University of Chittagong

Declaration

I hereby declare that the work presented in this thesis, entitled *"Lightweight CNN Architecture for Bangla Handwritten Digit Recognition"*, is the outcome of my own investigation carried out under the supervision of Dr. Iqbal Ahmed, Department of Computer Science and Engineering, University of Chittagong. This thesis is submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Engineering in Computer Science and Engineering.

I further declare that:

- The work reported here has not been submitted, in whole or in part, for any other degree or professional qualification at this or any other institution.
- Wherever the work of other researchers has been used, whether in the form of published articles, datasets, source code, figures or results, it has been explicitly acknowledged and correctly cited in the text and in the list of references.
- The dataset used in this study, NumtaDB, is a publicly released benchmark corpus of Bangla handwritten digits, and it has been used strictly in accordance with the terms under which it was made available.
- All quantitative results, tables and figures reported in Chapter 5 were produced by the experiments described in Chapter 4, and no result has been reproduced from another source without attribution.

Md. Johirul Islam
Student ID: 21701018
Session: 2020-2021

Date: _____

Acknowledgement

First and foremost, I express my sincere gratitude to the Almighty for granting me the patience, health and perseverance required to bring this research to completion.

I am deeply indebted to my supervisor, Dr. Iqbal Ahmed, Professor, Department of Computer Science and Engineering, University of Chittagong, for the guidance, technical insight and generosity of time that shaped this work from an initial idea into a completed thesis. The critical questions raised during our discussions repeatedly redirected the research towards more defensible ground, particularly in relation to the design of the ablation-free lightweight architecture and the interpretation of the validation instability reported in Chapter 5. Whatever methodological rigour this thesis possesses owes a great deal to that supervision.

I gratefully acknowledge the faculty members of the Department of Computer Science and Engineering, University of Chittagong, whose teaching over the course of the undergraduate programme provided the foundation in linear algebra, probability, digital image processing and machine learning on which this thesis depends. I also thank the departmental technical staff for maintaining the computing facilities that supported the early stages of experimentation.

This research would not have been possible without the NumtaDB corpus. I therefore record my thanks to the assemblers of that dataset, and to the several thousand anonymous contributors whose handwriting samples constitute it. Public benchmark datasets of this kind are a genuine public good in low-resource language research, and the willingness of their creators to release them openly is what makes reproducible work in Bangla document analysis possible at all.

I thank my fellow students for the many informal conversations that helped clarify my thinking, for reviewing early drafts of this manuscript, and for contributing samples of their own handwriting to the informal out-of-distribution test described in Section 5.5.

Finally, and most importantly, I thank my family for their unwavering patience and support throughout my studies. Their encouragement sustained this work through its more difficult phases, and this thesis is dedicated to them.

Md. Johirul Islam
August 2026

Abstract

Handwritten digit recognition is a foundational problem in document image analysis, and it underpins practical systems for postal automation, bank cheque processing, census form digitisation, examination-script handling and archival transcription. For the Bangla script, however, the problem remains substantially harder than for Latin numerals. Bangla contains digit pairs whose canonical forms differ only in small topological details, its handwriting exhibits wide regional and stylistic variation, and the corpora available for training are noisier and less curated than long-established Latin benchmarks. A further and increasingly important difficulty is that much of the reported progress in Bangla handwritten digit recognition has been obtained with very deep or heavily ensembled convolutional networks whose parameter counts and inference costs place them beyond the reach of the low-cost mobile and embedded devices on which realistic deployment in Bangladesh would actually occur.

This thesis addresses that gap by designing, training and evaluating a deliberately lightweight convolutional neural network for Bangla handwritten digit recognition. The proposed architecture replaces standard convolution throughout its feature extractor with depthwise separable convolution, factorising each spatial filtering operation into a depthwise stage that filters channels independently and a pointwise stage that recombines them. Four such blocks, each augmented with batch normalisation, rectified linear activation, max pooling and an additive residual connection, are followed by global average pooling, a single narrow fully connected layer with dropout, and a ten-way softmax classifier. The elimination of large flattened dense layers in favour of global average pooling is central to the design: it is the single change that most reduces parameter count while simultaneously acting as a structural regulariser. The complete model contains 248,586 parameters, of which 244,618 are trainable, corresponding to a stored footprint of roughly one megabyte in single-precision floating point.

The model was trained and evaluated on NumtaDB, a large assembled corpus of Bangla handwritten digits. After preprocessing, which included conversion to grayscale, resizing to 32×32 pixels, intensity normalisation to the unit interval, and the removal of 18 exact duplicate images, the working dataset contained 72,027 unique samples. These were partitioned by stratified sampling into 50,418 training, 10,804 validation and 10,805 test images, preserving the class distribution across all three subsets. Training used the Adam optimiser with sparse categorical cross-entropy, geometric data augmentation comprising random rotation, width and height shift and zoom, and a callback regime combining early stopping, adaptive learning-rate reduction on plateau and best-checkpoint restoration. Training terminated after 27 epochs.

On the held-out test partition the proposed model attained a test accuracy of 98.5007% with a test loss of 0.0525, corresponding to 10,643 correct predictions out of 10,805. Macro-averaged and weighted-averaged precision reached 0.9851 and macro-averaged recall and F1-score reached 0.9850, confirming that performance is balanced rather than concentrated in the easier classes. Per-class F1-scores ranged from 0.9704 to 0.9944. A detailed analysis of the confusion matrix shows that the 162 residual errors are not uniformly distributed but instead concentrate in a small number of script-motivated confusion pairs: 27 instances of digit six predicted as three, 25 instances of nine predicted as one, and 19 instances

of one predicted as nine together account for slightly over 43% of all errors. These are precisely the pairs whose Bangla glyphs share dominant strokes and differ mainly in the presence, curvature or closure of a secondary feature, and they are therefore interpretable as genuine script ambiguity rather than as arbitrary model failure. The analysis also documents and explains a pronounced epoch-to-epoch oscillation in validation loss during the high-learning-rate phase of training, and shows how the learning-rate reduction schedule progressively suppressed it.

The contribution of this work is therefore threefold. It demonstrates that competitive Bangla handwritten digit recognition accuracy is attainable at a parameter budget roughly an order of magnitude below that of the deep architectures most frequently reported for this task. It provides a transparent, fully specified and reproducible training and evaluation protocol, including explicit duplicate removal, which is a step often omitted in work on assembled corpora and one that can otherwise inflate reported accuracy. And it supplies a class-level error analysis that links the model's residual mistakes to identifiable structural properties of the Bangla numeral system, providing concrete direction for future work on confusion-aware training objectives and deployment-oriented compression.

Table of contents

List of figures

List of tables

Abbreviations

ANN	Artificial Neural Network
Adam	Adaptive Moment Estimation
API	Application Programming Interface
AUC	Area Under the Curve
BHDR	Bangla Handwritten Digit Recognition
BN	Batch Normalisation
CMATERdb	Centre for Microprocessor Applications for Training, Education and Research database
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DBN	Deep Belief Network
DSC	Depthwise Separable Convolution
DW	Depthwise (convolution)
FLOPs	Floating Point Operations
FN	False Negative
FP	False Positive
GAP	Global Average Pooling
GPU	Graphics Processing Unit
HOG	Histogram of Oriented Gradients
ILSVRC	ImageNet Large Scale Visual Recognition Challenge
KNN	k-Nearest Neighbours
LR	Learning Rate
LSTM	Long Short-Term Memory
MAC	Multiply–Accumulate operation
MLP	Multilayer Perceptron
MNIST	Modified National Institute of Standards and Technology database
NumtaDB	Assembled Bengali Handwritten Digits database
OCR	Optical Character Recognition
PCA	Principal Component Analysis
PW	Pointwise (convolution)
ReLU	Rectified Linear Unit
RGB	Red, Green, Blue
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
SGD	Stochastic Gradient Descent
SIFT	Scale-Invariant Feature Transform
SVM	Support Vector Machine
TFLite	TensorFlow Lite
TN	True Negative
TP	True Positive

Chapter 1

Introduction

1.1 Background

The automatic recognition of handwriting remains a fundamental problem in pattern recognition due to the large variation caused by differences in writing style, instruments, surfaces, acquisition devices, and environmental conditions. Handwritten digit recognition has been widely used as a benchmark task because digits represent a fixed set of classes while still presenting significant classification challenges [21]. It has applications in document processing, cheque recognition, postal automation, and digital archiving, where accurate recognition can improve efficiency and reduce manual effort.

Although handwritten digit recognition for Latin scripts has achieved near-human performance on benchmark datasets such as MNIST, this progress does not represent a universal solution for all writing systems. Many languages still lack sufficient datasets and mature recognition models. Bangla (Bengali), one of the world's most widely spoken languages, remains comparatively underdeveloped in handwriting recognition research despite its importance in Bangladesh and parts of India. The limited availability of large annotated datasets and efficient recognition models creates a significant gap between the need for Bangla handwriting recognition systems and existing technological capabilities.

1.1.1 The Bangla Numeral System and Its Recognition Challenges

The Bangla numeral system is decimal and positional, using ten glyphs derived historically from the Brahmi numerals and shared, with variation, across several Indic scripts. Four properties distinguish it from the Latin case in ways that matter directly for recognition.

The first is the prominence of curved, looped and enclosed strokes. Where Latin numerals are dominated by straight segments and open curves, several Bangla numerals contain closed or nearly closed loops whose presence or absence is the primary cue separating one digit from another. Loop closure is precisely the feature most vulnerable to degradation: a hurried writer may leave a loop open, a heavy pen may fill it in, and a low-resolution scan may destroy the few pixels involved. Bangla recognition therefore rests on features inherently less stable under real acquisition conditions than those distinguishing Latin numerals. The second property is that certain digit pairs are structurally similar by construction, differing only in a secondary feature such as the curvature of a terminal stroke or the presence of a small hook. This similarity is present in the printed forms themselves, and handwriting compounds it, because the distinguishing secondary feature is the one most likely to be abbreviated when writing quickly. Chapter 5 shows empirically that the residual errors of a well-trained model concentrate almost entirely in a small number of such pairs, which is direct evidence that these confusions are intrinsic to the script rather than incidental to any particular model.

The third property is pronounced regional and generational variation in letterforms, which extends to the numerals and is present within the training data itself, since assembled corpora aggregate contributions from varying populations and protocols. This inflates

within-class variance and makes the decision boundaries correspondingly harder to learn. The fourth is a matter of data rather than script: Bangla corpora are typically assembled from multiple independently collected sources, which is what makes large corpora feasible in a low-resource setting but introduces heterogeneity in resolution, background colour, contrast polarity, noise and marginal padding, and admits the possibility of exact or near-exact duplicate images. Duplicates spanning the training and test partitions contaminate reported accuracy with memorisation and overstate true generalisation, a hazard that Chapter 3 addresses directly.

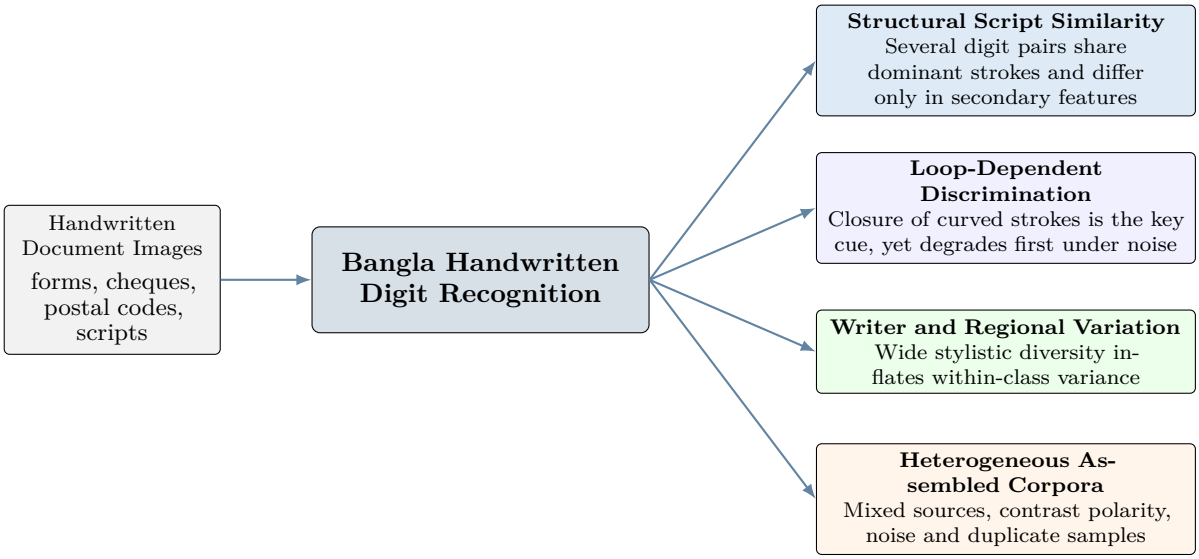

Fig. 1.1 Background of the study: the four principal sources of difficulty that distinguish Bangla handwritten digit recognition from the corresponding task in the Latin script.

1.1.2 The Efficiency Problem and Its Relevance to Bangla

The trajectory just described created a difficulty that is central to the motivation of this thesis. The models that achieve the highest reported accuracies on handwritten digit benchmarks, including Bangla benchmarks, are frequently very large. They may contain tens of millions of parameters, they may be ensembles of several such networks, or they may be transfer-learned adaptations of architectures originally designed for thousand-class natural image classification. Applied to a ten-class problem on 32×32 grayscale inputs, such models are enormously overprovisioned. Their capacity greatly exceeds what the task requires, and the accuracy advantage they obtain over a well-designed compact model is typically small, often a fraction of a percentage point.

The costs of that overprovisioning, however, are not small. A model with tens of millions of parameters occupies tens or hundreds of megabytes of storage, requires correspondingly large working memory during inference, consumes significant energy per prediction, and exhibits latency that may preclude interactive use on modest hardware. These costs are decisive precisely in the deployment settings where Bangla handwritten digit recognition would be most valuable. A postal sorting facility, a rural bank branch, a district education office digitising examination scripts, or a field enumerator capturing census returns on a low-cost Android handset are all environments characterised by constrained compute, constrained memory, intermittent or absent network connectivity, and constrained budgets. A recognition model that can only run in a datacentre is, for these purposes, not a solution.

This observation motivated a distinct line of architectural research concerned with

efficiency rather than raw accuracy. The most influential idea to emerge from it is depthwise separable convolution, popularised by the MobileNet family [16, 28] and given a thorough theoretical treatment in Xception [7]. The technique factorises a standard convolution into two simpler operations: a depthwise stage that applies one spatial kernel per input channel independently, and a pointwise stage of 1×1 convolutions that recombines the resulting channels. As Chapter 4 derives formally, this factorisation reduces both parameters and multiply–accumulate operations by a factor of approximately $1/N + 1/K^2$, where N is the number of output channels and K the spatial kernel size. For the 3×3 kernels used throughout this work the reduction is close to eightfold or ninefold. Complementary techniques include the replacement of flattened dense layers by global average pooling [**?**], which removes what is typically the largest single parameter block in a classification network, and channel shuffling or grouped convolution as used in ShuffleNet [**?**].

The central hypothesis of this thesis is that these efficiency techniques, combined thoughtfully rather than applied mechanically, are sufficient to build a Bangla handwritten digit classifier that is competitive in accuracy with far larger models while remaining small enough for practical deployment on commodity mobile hardware. The results reported in Chapter 5 support that hypothesis: the proposed model attains 98.5007% test accuracy with 248,586 total parameters.

1.2 Motivation

Four considerations motivate this research.

Deployment realism. Research accuracy is only useful if it can be delivered where the problem occurs. In Bangladesh, the plausible deployment targets for a Bangla digit recogniser are mid-range and low-end smartphones, embedded scanning devices, and modest desktop machines in offices without dedicated accelerators. Reported state-of-the-art models are, with few exceptions, unsuitable for these targets. A model of roughly one megabyte that runs comfortably within a mobile inference runtime addresses a real and currently unmet need, and it does so without requiring the network connectivity that a server-side alternative would demand.

Methodological hygiene in assembled corpora. Large Bangla handwriting corpora are assembled from multiple contributed sources. Such assembly can introduce duplicate images. If duplicates straddle the training and evaluation partitions, a model can achieve apparently excellent test performance partly by recall rather than by generalisation. Explicit duplicate detection and removal before partitioning is therefore not a minor preprocessing detail but a precondition for trustworthy measurement. This step is frequently unreported in the literature. This thesis performs it explicitly, documents it, and reports the resulting corpus size.

Interpretability of residual error. A single aggregate accuracy figure conveys very little about a classifier's behaviour. Two models with identical accuracy may fail in entirely different ways, one distributing errors uniformly and the other concentrating them in a few systematic confusions. The latter is far more informative, because concentrated errors point to identifiable causes and therefore to actionable remedies. This thesis treats the confusion matrix as a primary object of analysis rather than a summary artefact, and shows that the model's errors localise in script-motivated digit pairs.

Contribution to low-resource script research. Techniques validated on Bangla are likely to transfer to other Indic and low-resource scripts that share structural characteristics and face similar resource constraints. Establishing that a compact architecture suffices for Bangla provides evidence relevant well beyond the specific language.

1.3 Problem Statement

The problem addressed in this thesis can be stated precisely as follows.

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{M}$ denote a corpus of labelled Bangla handwritten digit images, where each $\mathbf{x}_i$ is an image of a single isolated digit and each label $y_i \in \mathcal{Y} = \{0, 1, \ldots, 9\}$ identifies the digit's numeric value. After preprocessing, each image is represented as a tensor $\mathbf{x} \in \mathbb{R}^{32\times32\times1}$ whose entries lie in $[0, 1]$. The objective is to learn a parametric classifier

$$f_{\boldsymbol{\theta}} : \mathbb{R}^{32\times32\times1} \to \Delta^9, \qquad \Delta^9 = \Big\{ \mathbf{p} \in \mathbb{R}^{10} \;\Big|\; p_k \geq 0,\ \textstyle\sum_{k=0}^{9} p_k = 1 \Big\}, \tag{1.1}$$

mapping an image to a probability distribution over the ten digit classes, with the predicted label obtained as $\hat{y} = \arg\max_{k\in\mathcal{Y}} \big[f_{\boldsymbol{\theta}}(\mathbf{x})\big]_k$.

The learning objective is the minimisation of expected categorical cross-entropy over the data distribution, approximated empirically on the training partition:

$$\boldsymbol{\theta}^{\star} = \arg\min_{\boldsymbol{\theta}} \; \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}_i, y_i)\in\mathcal{D}_{\text{train}}} -\log\big[f_{\boldsymbol{\theta}}(\mathbf{x}_i)\big]_{y_i}. \tag{1.2}$$

What distinguishes this thesis from a routine application of that objective is the imposition of an explicit capacity constraint. The problem is not merely to maximise accuracy but to maximise it subject to

$$|\boldsymbol{\theta}| \;\ll\; |\boldsymbol{\theta}_{\text{deep}}|, \tag{1.3}$$

where $|\boldsymbol{\theta}_{\text{deep}}|$ denotes the parameter count of the deep architectures conventionally applied to this task. Concretely, the design target adopted here is a total parameter count below three hundred thousand, which corresponds to a stored model of roughly one megabyte in single-precision floating point and is therefore compatible with mobile deployment.

The research problem is thus the joint design of an architecture, a preprocessing and augmentation pipeline, and a training protocol that together satisfy this constraint without material loss of accuracy, together with an evaluation methodology sufficiently detailed to characterise where and why the resulting model fails.

1.3.1 Research Questions

The thesis is organised around five research questions.

1. **RQ1.** Can a convolutional architecture built from depthwise separable convolution, residual connections and global average pooling, and constrained to fewer than three hundred thousand parameters, achieve Bangla handwritten digit recognition accuracy competitive with substantially larger conventional networks?
2. **RQ2.** What quantitative reduction in parameters and multiply–accumulate operations does depthwise separable convolution deliver relative to standard convolution at the specific layer configurations used in the proposed model, and how is that

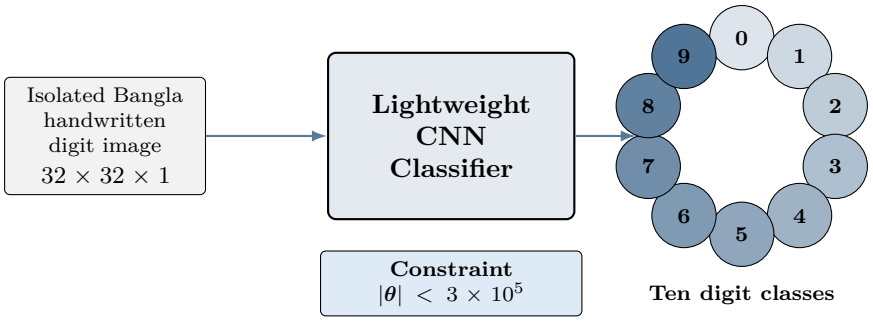

Fig. 1.2 Problem statement: mapping a preprocessed isolated Bangla handwritten digit image to one of ten classes, subject to an explicit parameter-budget constraint that makes on-device deployment feasible.

reduction distributed across the network?

3. **RQ3.** How is the residual error of the trained model distributed across the ten digit classes, and do the dominant confusions correspond to identifiable structural similarities in the Bangla numeral glyphs rather than to arbitrary model failure?
4. **RQ4.** What effect do the components of the training protocol, specifically geometric augmentation, adaptive learning-rate reduction and best-checkpoint restoration, have on the stability of optimisation and on the gap between validation and test performance?
5. **RQ5.** Does the model's performance on isolated, curated benchmark images transfer to photographed handwriting captured outside the dataset's acquisition conditions, and what does any observed degradation reveal about the limits of the training distribution?

Questions RQ1 and RQ2 are addressed by the architectural analysis of Chapter 4 together with the accuracy results of Section 5.2. RQ3 is addressed by the confusion matrix and class-wise analysis of Sections 5.3 and 5.4. RQ4 is addressed by the training dynamics analysis of Section 5.1. RQ5 is addressed by the out-of-distribution prediction study of Section 5.5.

1.4 Research Challenges

Five categories of challenge shape the work reported here.

Script-intrinsic ambiguity. Certain Bangla digit pairs are structurally similar by construction, sharing dominant strokes and differing only in secondary features. No amount of model capacity eliminates this ambiguity, because in the limiting cases the ambiguity is present in the image itself, and a human reader would rely on surrounding context that an isolated-digit classifier does not have. The realistic goal is therefore not elimination but containment: ensuring that confusions occur only in the genuinely ambiguous tail rather than across clearly distinguishable samples.

Corpus heterogeneity. An assembled corpus aggregates sources that differ in resolution, contrast polarity, background texture, marginal padding and noise characteristics. Preprocessing must normalise this heterogeneity sufficiently for learning to proceed, without discarding the diversity that gives the corpus its value as a proxy for real-world variation.

The balance is delicate: over-aggressive normalisation produces a model that performs well on the corpus and poorly in the field.

The capacity–accuracy tension. The parameter budget is a hard constraint, and every architectural choice must be justified against it. Reducing capacity too far causes underfitting, in which the model cannot represent the decision boundaries the task requires. Retaining capacity in the wrong place, most commonly in flattened dense layers, wastes budget on parameters that contribute little. Resolving this tension requires understanding where in a network parameters actually reside, which is why Chapter 4 presents a layer-by-layer parameter accounting.

Optimisation stability. Training a batch-normalised network with an adaptive optimiser at a relatively high initial learning rate, on augmented data, produces validation metrics that can oscillate violently between consecutive epochs. This behaviour is well documented but remains disconcerting, and it raises a genuine methodological question: if validation accuracy fluctuates by tens of percentage points from one epoch to the next, which epoch's model should be retained? The training protocol must answer this question in a principled way rather than by arbitrary selection. Section 5.1 analyses this phenomenon in detail using the observed training history.

Generalisation beyond the corpus. A model evaluated only on a held-out partition of the same corpus it was trained on is evaluated under optimistic conditions, because training and test images share acquisition characteristics. Photographed handwriting on ordinary paper, captured under uncontrolled lighting with a phone camera, differs systematically from curated corpus images. Assessing performance under these conditions requires an additional preprocessing pipeline and reveals limitations invisible in standard evaluation.

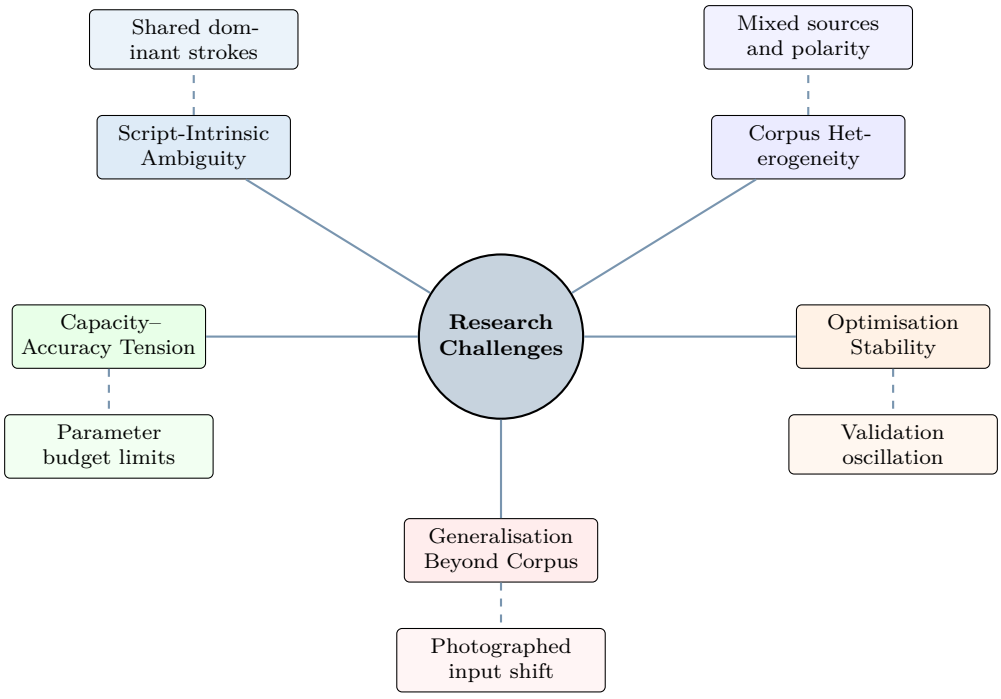

Fig. 1.3 The five categories of research challenge addressed in this thesis and their principal manifestations.

1.5 Scope of the Study

The boundaries of this study are defined as follows.

Included. The work addresses the classification of *isolated* Bangla handwritten digits, that is, images each containing a single digit that has already been located and cropped. The dataset is NumtaDB, used in its publicly released form. Images are converted to grayscale, resized to 32×32 pixels and normalised to the unit interval. Exact duplicates are detected and removed before partitioning. The model is a convolutional neural network built from depthwise separable convolution blocks with batch normalisation, rectified linear activation, max pooling and additive residual connections, terminating in global average pooling, one dense hidden layer with dropout, and a ten-way softmax. Training uses the Adam optimiser with sparse categorical cross-entropy and geometric augmentation. Evaluation reports accuracy, loss, the full confusion matrix, per-class precision, recall, F1-score and support, and macro- and weighted-averaged summaries. A supplementary qualitative assessment applies the model to photographed handwriting.

Excluded. Several related problems lie outside the scope. Digit *detection* and *segmentation* from full document pages are not addressed; the input is assumed pre-segmented. Recognition of Bangla *characters*, *compound characters* and *words* is not addressed, as these involve substantially larger label spaces and, for words, sequence modelling. Multi-digit *numeral strings* are not addressed. The study does not perform *quantisation*, *pruning* or *knowledge distillation*, although these are identified as future work; the efficiency claims rest on architectural design alone. The study does not construct a deployed mobile application, does not perform on-device latency benchmarking, and does not conduct a formal ablation study isolating the individual contribution of each architectural component. Finally, no new dataset is collected; the contribution is methodological rather than resource-creating.

1.6 Research Contributions

This thesis makes the following contributions.

1. **A parameter-efficient architecture for Bangla handwritten digit recognition.** A four-block convolutional network is proposed in which every spatial filtering operation is realised as a depthwise separable convolution, each block carries an additive residual connection, and classification is performed after global average pooling rather than flattening. The complete model contains 248,586 parameters, of which 244,618 are trainable and 3,968 are the non-trainable running statistics of the batch normalisation layers. The design achieves 98.5007% test accuracy at this budget.
2. **A formal efficiency analysis of the specific architecture.** Rather than citing generic efficiency claims, the thesis derives the parameter and multiply–accumulate cost of each layer of the proposed model and compares it against the standard-convolution equivalent, quantifying the saving block by block. This makes the source of the efficiency gain explicit and answers RQ2 concretely.
3. **A reproducible preprocessing protocol including explicit duplicate removal.** The thesis documents the removal of 18 exact duplicate images prior to partitioning, yielding a working corpus of 72,027 unique samples, and applies stratified sampling

to produce 50,418 training, 10,804 validation and 10,805 test images. Reporting this step explicitly addresses a methodological gap in work on assembled corpora.

4. **A detailed, script-grounded error analysis.** The thesis analyses the full 10×10 confusion matrix and demonstrates that the 162 residual errors concentrate in a small number of digit pairs, with the three dominant confusions accounting for slightly over 43% of all errors. It further shows that these pairs correspond to structurally similar Bangla glyphs, establishing that the errors are script-motivated rather than arbitrary.
5. **An analysis of optimisation dynamics under adaptive learning-rate scheduling.** Using the recorded training history, the thesis documents pronounced validation oscillation during the high-learning-rate phase, explains its mechanism in terms of batch normalisation statistics and optimiser step size, and shows how successive learning-rate reductions suppressed it, with validation loss falling from a peak of 3.4994 to a final 0.0525.
6. **An out-of-distribution assessment on photographed handwriting.** The thesis reports a qualitative evaluation on photographed samples processed through a dedicated pipeline of grayscale conversion, thresholding, centring and resizing, documenting both a high-confidence success and a low-confidence case, and drawing conclusions about the limits of the training distribution.

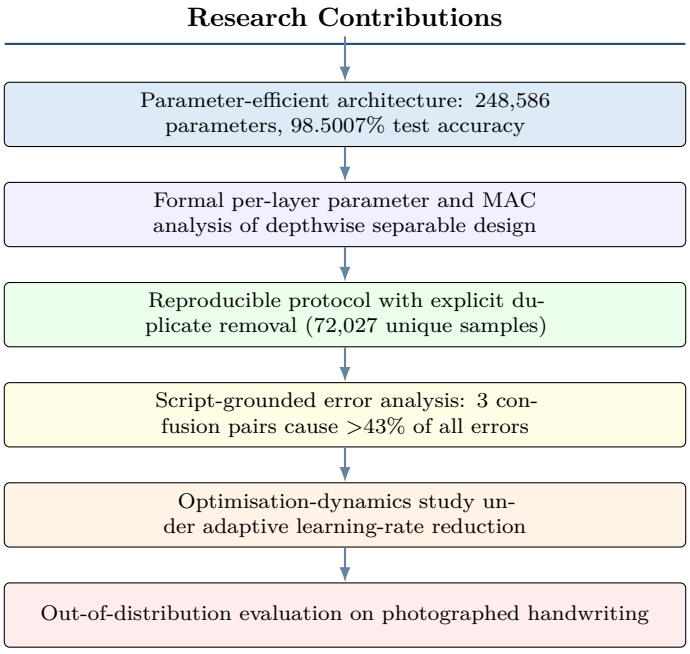

Fig. 1.4 Summary of the principal research contributions of this thesis.

1.7 Thesis Organization

The remainder of this thesis is organised into five further chapters.

Chapter 2: Related Work surveys the literature relevant to this study. It traces the development of handwritten character and digit recognition from classical pipelines through to modern convolutional architectures, examines the specific literature on Bangla handwritten digit recognition and the datasets that support it, reviews the family of lightweight architectures from which the proposed design draws, and treats depthwise

separable convolution and residual learning in the depth their centrality warrants. The chapter closes by articulating the research gap that this thesis addresses.

Chapter 3: Dataset and Data Preparation describes NumtaDB, its provenance, structure and class organisation. It documents the duplicate detection and removal procedure that reduced the corpus to 72,027 unique images, specifies the preprocessing pipeline, describes the geometric augmentation strategy, discusses the challenges the dataset presents, and reports the stratified partition into training, validation and test subsets.

Chapter 4: Proposed Methodology presents the architecture in full. It develops the mathematics of standard convolution, depthwise convolution, pointwise convolution and their separable composition, deriving the resulting efficiency gain. It then explains batch normalisation, rectified linear activation, max pooling, residual connections, global average pooling, dropout and softmax classification, before assembling these components into the complete network and giving a layer-by-layer parameter accounting. The chapter concludes with the training configuration and the definitions of the evaluation metrics.

Chapter 5: Results and Evaluation reports and interprets the experimental outcomes. It analyses the training and validation accuracy and loss trajectories across all 27 epochs, reports test performance, presents and analyses the confusion matrix in detail, examines per-class precision, recall and F1-score, reports the out-of-distribution prediction study, positions the results against prior work, and discusses their implications.

Chapter 6: Conclusion and Future Work summarises the research, restates the answers to the five research questions, states the limitations of the study candidly, and identifies directions for subsequent work including quantisation, confusion-aware training objectives, and extension to the broader Bangla character set.

Chapter 2

Related Work

Handwritten digit recognition has a research history spanning several decades, and the trajectory of that history mirrors the broader evolution of pattern recognition itself, from handcrafted feature engineering through classical statistical learning to deep representation learning. This chapter reviews the literature that situates the present thesis. It begins with a general overview of handwritten recognition and the classical pipeline paradigm, proceeds through traditional machine learning approaches and the deep learning transition, examines convolutional methods in detail, then narrows to the specific literature on Bangla handwritten digit recognition and the datasets that support it. It subsequently reviews the family of lightweight architectures and treats depthwise separable convolution and residual learning at length, since these two ideas are the load-bearing elements of the proposed design. The chapter concludes by articulating the research gap.

2.1 Overview of Handwritten Recognition

Handwritten recognition is commonly classified along two axes. The first distinguishes *online* and *offline* recognition. Online recognition uses the temporal trajectory of pen movement, including stroke order, direction, and velocity, whereas offline recognition relies only on the final image of the written content. This thesis focuses on offline recognition, which is more challenging because dynamic stroke information is unavailable [26]. The second axis concerns the recognition target, ranging from isolated characters and digits to words, text lines, and full pages. Although isolated digit recognition has a smaller label space, it remains challenging for scripts such as Bangla due to structural similarities between digit classes.

The field of handwritten digit recognition was strongly influenced by the MNIST database, consisting of 70,000 normalised 28×28 grayscale images of handwritten Latin digits [21]. MNIST provided a common benchmark and demonstrated the effectiveness of convolutional neural networks, with advanced approaches achieving error rates below half a percent [8]. However, its success also highlighted the gap between well-resourced Latin scripts and low-resource scripts, as many evaluation practices assume clean and balanced datasets that may not represent real-world heterogeneous corpora [12].

For Indic scripts, dataset development progressed more slowly. Bhattacharya and Chaudhuri [4] introduced handwritten numeral databases including Bangla, while broader surveys highlighted the limited availability of annotated resources for Indic script recognition [25]. This data scarcity remains a major challenge and motivates the development of effective recognition models for Bangla handwritten numerals.

2.2 Traditional Machine Learning Approaches

Before the deep learning era, offline handwritten digit recognition was mainly based on a pipeline approach consisting of preprocessing, handcrafted feature extraction, and

classification. The primary research focus was the design of effective feature representations.

Handcrafted features can be broadly classified into structural, statistical, and transform-based descriptors. Structural features represent the geometry of glyphs through contours, skeletons, endpoints, junctions, and loops. Statistical features describe ink distribution using zoning, projection profiles, and crossing counts. Transform-based methods extract compact representations using techniques such as discrete cosine transform, wavelets, Gabor filters, and moment-based descriptors including Hu invariants [18]. General computer vision descriptors such as histogram of oriented gradients (HOG) [10] and scale-invariant feature transform (SIFT) [23] were also applied to handwriting recognition.

The extracted features were classified using traditional machine learning methods. k-nearest neighbours provided a strong baseline for well-normalised digit datasets, while support vector machines became one of the most effective classifiers due to their maximum-margin optimisation and kernel-based feature mapping [9]. Other approaches included multilayer perceptrons and ensemble methods such as random forests [6].

However, the major limitation of this paradigm was that feature extraction was manually designed and fixed before learning. Such representations often failed to generalise across different scripts and acquisition conditions. This limitation was particularly significant for Bangla handwriting, where complex stroke patterns, loops, and enclosed regions require script-specific feature design. As discussed in statistical learning theory [5], fixed representations introduce bias that cannot be fully eliminated through classifier optimisation alone, motivating the shift towards data-driven feature learning approaches.

2.3 Deep Learning Approaches

The transition to learned representations removed the frozen-representation limitation at a stroke. A deep neural network trained end to end optimises its internal features against the classification objective, so the representation adapts to whatever data and script it is trained on. This adaptivity is precisely why deep learning transferred across scripts far more readily than handcrafted pipelines: the same architecture, retrained, learns Bangla-appropriate features without any redesign. The conceptual foundations of this shift, and the representation-learning perspective that underpins it, are surveyed by LeCun et al. [22] and treated at length by Goodfellow et al. [13].

Several architectural families populate the deep learning literature on handwriting. Deep belief networks and stacked autoencoders, trained by greedy unsupervised pretraining, were influential in the transitional period before purely supervised training of deep networks became routine [**?**]. Recurrent architectures, especially the long short-term memory network [**?**], are central to *sequence* recognition of words and lines, where they are combined with connectionist temporal classification to avoid explicit character segmentation, but they are less natural for isolated single-digit classification, which lacks a meaningful temporal axis in the offline setting. For the isolated-digit problem addressed here, the convolutional neural network is both the historically dominant and the empirically superior choice, and it is treated separately below.

The generic advantages of deep models, high representational capacity, end-to-end optimisation, and hierarchical feature abstraction, come with generic costs that are the central concern of this thesis: large parameter counts, substantial training-data requirements, significant computational demand, and a propensity to overfit that must be countered by regularisation. The lightweight design philosophy pursued here is best understood as an attempt to retain the first set of advantages while sharply curtailing the associated costs.

2.4 CNN-Based Methods

The convolutional neural network (CNN) is based on three key principles: local receptive fields, weight sharing, and spatial subsampling. These concepts introduce locality and translation equivariance while significantly reducing parameters compared with fully connected networks. LeNet-5 established the fundamental convolution–subsampling architecture that remains the foundation of modern CNNs [21].

The modern development of CNNs accelerated after the success of deep networks on large-scale image classification tasks [20]. Subsequent architectures improved performance through increased depth, efficient feature extraction, and better optimisation strategies. VGG networks demonstrated that stacking small 3×3 kernels could achieve strong representations with fewer parameters and increased non-linearity [33]. Inception introduced multi-scale feature extraction through parallel convolutional branches [36]. Batch normalisation improved training stability by controlling activation distributions [**?**], while residual learning enabled the training of deeper networks through identity-based shortcut connections [14]. Further developments, including DenseNet's feature reuse mechanism [19] and lightweight attention modules [17], improved feature learning efficiency.

For handwritten digit recognition, CNNs and their ensembles have achieved near-saturated performance on Latin-script benchmarks [8]. However, for low-resolution, ten-class classification tasks, increasing model depth and width provides diminishing accuracy improvements while increasing computational cost. This observation motivates the efficiency-oriented CNN design adopted in this thesis.

2.5 Bangla Handwritten Digit Recognition Studies

Research on Bangla handwritten digit recognition has expanded significantly with the availability of dedicated datasets. Early studies mainly used CMATERdb and ISI Bangla handwriting datasets, applying handcrafted features such as shadow features, longest-run features, and quad-tree descriptors with classical classifiers [11]. These approaches achieved high accuracy on relatively small datasets but were limited by the dependence on manually designed features and restricted data availability.

The adoption of convolutional neural networks (CNNs) gradually replaced handcrafted pipelines. CNN-based approaches trained on CMATERdb and related datasets achieved improved performance over traditional methods [31]. During the transition period, hybrid approaches combined handcrafted descriptors with deep models; for example, Sharif et al. [29] integrated histogram-of-oriented-gradients features with deep learning to improve Bangla numeral classification. These studies demonstrated the gradual shift from manually engineered representations to automatically learned features.

Fully deep learning approaches later became dominant. Alom et al. [2] investigated different convolutional architectures for Bangla digit recognition, while Alom et al. [3] compared modern architectures including VGG, residual, densely connected, and inception-based networks. These studies confirmed that architectures originally developed for natural images could transfer effectively to Bangla recognition. However, most evaluations focused primarily on accuracy without considering model size or computational cost. Rabby et al. [27] introduced BornoNet, a task-specific CNN for Bangla character recognition, while Sufian et al. [35] proposed BDNet, a densely connected architecture evaluated on NumtaDB with high recognition accuracy.

A major milestone was the release of NumtaDB, a large-scale Bangla handwritten

digit dataset containing approximately eighty-five thousand images collected from multiple sources [1]. Its diverse acquisition conditions made it a valuable benchmark but also introduced challenges such as data heterogeneity and possible labeling noise. Several studies using NumtaDB, including Shawon et al. [30] and Zunair et al. [38], achieved strong performance through deep CNNs and transfer learning. Nevertheless, most existing approaches prioritised accuracy using large backbones or ensembles, while reporting limited information about model complexity, inference cost, and duplicate removal.

Table 2.1 presents representative studies in Bangla handwritten digit recognition. The literature indicates that high accuracy is often achieved using computationally expensive architectures, highlighting the need for efficient models that balance recognition performance with practical deployment constraints.

Table 2.1 Representative studies in Bangla handwritten digit recognition. The final column records whether the study reports model size or efficiency, illustrating how rarely this is treated as a first-class concern.

Study / era	**Dataset**	**Method**	**Reported acc.**	**Efficiency reported?**
Classical pipeline [11]	CMATERdb	GA feature selection + MLP/SVM	∼97%	No
Indic numeral DB [4]	ISI (multi-script)	Multistage mixed-numeral recognition	∼98%†	No
Hybrid HOG + deep [29]	CMATERdb	HOG features + deep model	∼99%†	No
CNN + autoencoder [31]	CMATERdb / ISI	Unsupervised pretraining + CNN	∼99%†	No
Deep CNN survey [3]	CMATERdb	VGG / ResNet / DenseNet comparison	>99%†	No
BornoNet [27]	Bangla chars	Task-specific CNN	∼98%†	No
Deep CNN [30]	NumtaDB	Deep convolutional network	†	No
Transfer learning [38]	Bengali numerals	Cross-domain transfer	†	No
BDNet [35]	NumtaDB	Densely connected CNN	>99%†	Partially
This thesis	NumtaDB (dedup.)	Lightweight DSC-CNN + residual + GAP	**98.50%**	**Yes (248,586 params)**

† Accuracies quoted from third-party studies are reported under differing partition protocols, and in particular differing treatment of duplicates and augmentation, and are therefore not directly comparable with one another or with this work. They are reproduced here to indicate the general performance regime, not as a controlled comparison.

2.6 Lightweight CNN Architectures

A distinct research programme, largely orthogonal to the pursuit of maximal accuracy, has concerned itself with the design of convolutional networks that achieve strong accuracy under tight computational budgets. This programme supplies the design vocabulary of the present thesis.

The SqueezeNet architecture demonstrated that AlexNet-level ImageNet accuracy could be attained with roughly fiftyfold fewer parameters, using "fire modules" that squeeze the channel dimension with 1×1 convolutions before expanding it [**?**]. The MobileNet family made depthwise separable convolution the organising principle of an entire network and introduced width and resolution multipliers as explicit knobs for trading accuracy against cost [16]; its second version added inverted residual blocks with linear bottlenecks, connecting the efficiency and residual-learning literatures [28], and a third applied neural architecture search together with hardware-aware objectives to refine the design further [15]. ShuffleNet combined grouped pointwise convolution with a channel-shuffle operation to restore inter-group information flow at very low cost [**?**]; its successor argued influentially that floating-point operation counts are a poor proxy for realised latency and proposed direct-measurement design guidelines instead [**?**], a caution directly relevant to the limitations acknowledged in Section 6.4 of this thesis. EfficientNet later formalised the joint scaling of depth, width and resolution through a compound coefficient [**?**], building on platform-aware architecture search [**?**]. Across this literature two structural ideas recur as the primary sources of saving: the factorisation of convolution via depthwise separable convolution, and the elimination of large dense layers, most cleanly achieved by global average pooling [**?**]. The architecture proposed in Chapter 4 is built from exactly these two ideas, augmented with residual connections for trainability.

A complementary body of work reduces model cost after training rather than by design. Deep compression combined pruning, trained quantisation and entropy coding to achieve order-of-magnitude size reductions without accuracy loss [**?**]; integer-arithmetic-only quantisation schemes made 8-bit inference practical on commodity mobile hardware [**?**]; and knowledge distillation transfers the behaviour of a large teacher network into a smaller student [**?**]. These techniques are orthogonal to architectural efficiency and compose with it. They were not applied in this thesis, whose efficiency claims rest on design alone, but they are identified in Section 6.5 as the most direct route to further compression of the proposed model.

Table 2.2 places the proposed model in the context of this lightweight lineage, emphasising which techniques each design employs.

2.7 Depthwise Separable Convolution

Because depthwise separable convolution is the central mechanism of the proposed architecture, its conceptual lineage warrants specific review; its mathematics is developed fully in Section 4.2.4.

The factorisation of a spatial convolution into a per-channel spatial stage followed by a cross-channel combination stage predates its modern popularity. Its earliest clear statement appears in work on rigid-motion scattering transforms, where separable filtering across spatial and channel axes was introduced on mathematical rather than computational grounds [32]. The idea appears again in the flattened convolutional networks of **?**] and in related work on low-rank factorisation of convolutional kernels, all motivated by the observation that a full convolution simultaneously mixes spatial and cross-channel information and

Table 2.2 Lightweight architectural techniques and their adoption. The proposed model combines depthwise separable convolution, global average pooling and residual connections in a deliberately shallow four-block design sized for a ten-class digit task.

Architecture	Depthwise sep. conv.	Global avg. pool	Residual	Primary target
SqueezeNet [**?**]	No (1×1 squeeze)	No	Later	ImageNet, small params
MobileNetV1 [16]	Yes	Yes	No	Mobile ImageNet
MobileNetV2 [28]	Yes	Yes	Yes (inverted)	Mobile ImageNet
ShuffleNet [**?**]	Yes (grouped)	Yes	Yes	Very low FLOPs
EfficientNet [**?**]	Yes	Yes	Yes	Compound scaling
Proposed	**Yes**	**Yes**	**Yes (additive)**	**On-device Bangla digits**

that these two functions can be separated at greatly reduced cost. Xception carried the idea to its logical extreme, replacing the Inception modules of GoogLeNet with depthwise separable convolutions throughout and interpreting the result as an assumption that spatial and cross-channel correlations in feature maps can be mapped entirely independently [7]. MobileNet then demonstrated that the same factorisation, combined with width and resolution multipliers, yields a family of models spanning a wide accuracy–efficiency spectrum suitable for mobile deployment [16, 28].

The consistent empirical finding across this literature is that the accuracy sacrificed by the factorisation is small, typically one to two percentage points on demanding thousand-class benchmarks, while the reduction in parameters and computation is large, typically eight- to ninefold for 3×3 kernels. On a ten-class digit task, where the representational demands are far lighter than on ImageNet, there is good reason to expect the accuracy sacrifice to be smaller still, perhaps negligible. The results of this thesis confirm that expectation.

2.8 Residual Learning

The second structural pillar of the proposed design is residual learning. The motivating observation of He et al. [14] was the counterintuitive *degradation* phenomenon: beyond a certain depth, adding further layers to a plain convolutional network increased not only test error but training error, which cannot be attributed to overfitting and instead reflects an optimisation difficulty. Residual learning resolves this by reformulating each block to compute a residual function $\mathcal{F}(\mathbf{x})$ added to its input, so that the block realises $\mathbf{x} + \mathcal{F}(\mathbf{x})$. The identity path provides an unobstructed route for gradient flow during backpropagation and makes the identity mapping trivially representable, so that additional blocks can, in the worst case, learn to do nothing rather than actively harming the function already learned. A closely contemporaneous proposal, highway networks, achieved a similar effect using learned gating functions to interpolate between the transformed and untransformed signal [34]; the residual formulation prevailed largely because it achieves comparable benefit with no additional parameters at all, which is an attractive property in the capacity-constrained setting of this thesis.

Although residual learning was devised for networks far deeper than the four-block model proposed here, its inclusion in a shallow network is nonetheless justified on two grounds developed in Section 4.3.4. First, the additive skip path improves gradient conditioning

even at modest depth, which in combination with batch normalisation permits stable training at the relatively high initial learning rate used in this work. Second, and more subtly, the skip connection allows each block to refine rather than replace the representation it receives, which suits a compact network in which every channel must be used efficiently. Later analysis characterised residual networks as behaving like ensembles of many shallower paths of varying length [37], a perspective that further explains why residual connections improve trainability without a commensurate increase in effective capacity, an attractive property when capacity is deliberately constrained.

2.9 Regularisation and Optimisation Techniques

The final body of literature relevant to this thesis concerns the techniques by which deep networks are trained and regularised, since several of these are components of the proposed method and one of them, batch normalisation, proves central to explaining the training behaviour reported in Chapter 5.

Initialisation and the gradient problem. The difficulty of propagating gradient signal through many layers was characterised early in the recurrent setting [**?**] and applies equally to deep feedforward networks. Principled initialisation schemes that preserve activation and gradient variance across layers substantially mitigated it, first for symmetric activations [**?**] and subsequently for rectified linear units, whose asymmetry requires a different scaling [**?**]. Rectified linear activation itself [**?**] was among the changes that made deep supervised training practical, by providing a non-saturating derivative.

Normalisation. Batch normalisation [**?**] was originally motivated as a remedy for internal covariate shift, though later analysis argued that its principal benefit is instead a smoothing of the optimisation landscape [**?**]. For the present thesis the mechanistically important point is one neither paper emphasises: the operation behaves differently in training and inference, using mini-batch statistics in the former and accumulated running estimates in the latter. Alternatives such as layer normalisation [**?**] avoid this train–inference asymmetry by normalising across features rather than across the batch, and are correspondingly less prone to the instability documented in Section 5.1.

Regularisation. Dropout [**?**] remains the most widely used explicit regulariser for dense layers, and variants such as DropConnect, which drops individual weights rather than activations [**?**], explore the same principle. Data augmentation provides an orthogonal and often more effective form of regularisation by enlarging the effective training distribution; the design space of image augmentation for deep learning is surveyed comprehensively by **?**], whose taxonomy informs the distinction drawn in Section 3.5 between the geometric transformations applied in this work and the photometric ones that were not.

Optimisation and scheduling. Adaptive optimisers, of which Adam [**?**] is the most widely adopted, adjust per-parameter step sizes from gradient moment estimates and are notably robust to hyperparameter choice; the broader theory of stochastic optimisation for machine learning is reviewed by **?**]. Batch size interacts with generalisation in ways that are not fully understood, with evidence that very large batches converge to sharper minima that generalise less well [**?**], a consideration behind the modest batch size of 64 used here. Learning-rate scheduling has been explored through cyclical schedules [**?**] and warm

restarts [**?**]; the plateau-triggered reduction employed in this thesis is a simpler adaptive alternative whose stabilising effect is quantified in Section 5.1. Early stopping, finally, is a long-established regulariser whose subtleties in the presence of noisy validation curves were analysed by **?**], and whose combination with best-checkpoint restoration proves decisive for the training run reported in this thesis.

2.10 Research Gap

The literature reviewed in this chapter supports four observations that, taken together, define the gap this thesis addresses.

First, Bangla handwritten digit recognition is a maturing field in which high accuracies have been reported, but the highest of these are obtained with large or ensembled convolutional models whose parameter counts and inference costs are documented rarely, if at all. The efficiency of a model is treated as an afterthought rather than a design objective, even though efficiency is the property that most directly governs whether a model can be deployed in the Bangladeshi settings where it would be used.

Second, the lightweight architecture literature has established a mature and well-validated set of techniques, principally depthwise separable convolution, global average pooling and residual connections, but this literature is oriented overwhelmingly towards large-scale natural image classification. Its application to low-resource script recognition, and specifically to Bangla numerals, is comparatively unexplored, and the extent to which its efficiency gains transfer to the far lighter demands of a ten-class digit task has not been carefully quantified for this setting.

Third, the methodological hazard of duplicate images in assembled corpora such as NumtaDB is acknowledged in principle but addressed explicitly only rarely. Studies that partition an assembled corpus without first removing exact duplicates risk reporting inflated test accuracy, yet the deduplication step and its effect on corpus size are seldom documented.

Fourth, evaluation in this literature is dominated by aggregate accuracy. Detailed, script-grounded analysis of *which* digits a model confuses, and *why* those particular confusions arise from the structure of the Bangla numeral glyphs, is scarce, even though such analysis is far more informative for future improvement than a single accuracy figure.

This thesis addresses all four points. It designs an explicitly lightweight architecture and reports its exact parameter count; it transfers and quantifies the efficiency techniques of the lightweight literature to the Bangla numeral task; it performs and documents explicit duplicate removal before partitioning; and it makes the confusion structure of the trained model a central object of analysis, linking it to the geometry of the script. The chapters that follow develop each of these contributions in turn.

Chapter 3

Dataset and Data Preparation

The empirical claims of this thesis rest entirely on the dataset used to train and evaluate the proposed model, and on the procedures applied to prepare that dataset. This chapter documents both in full. It describes the NumtaDB corpus and its provenance, sets out the ten-class label structure of the Bangla numeral system, details the duplicate detection and removal procedure that produced the working corpus of 72,027 unique images, specifies the preprocessing pipeline and the geometric augmentation strategy, discusses the challenges the dataset presents, and reports the stratified partition into training, validation and test subsets. The level of detail here is deliberate: the reproducibility of the results in Chapter 5 depends on it, and the explicit treatment of duplicate removal addresses one of the methodological gaps identified in Section 2.10.

3.1 The NumtaDB Dataset

NumtaDB is a large publicly released database of Bangla handwritten digits, assembled specifically to provide the Bangla document analysis community with a benchmark of a scale comparable to those long available for the Latin script [1]. Its name derives from the Bangla word for "digit". Prior to its release, research on Bangla numeral recognition relied on corpora such as CMATERdb and the numeral subsets of the ISI Bangla handwriting collection, which, while carefully constructed, contain on the order of a few thousand to a few tens of thousands of samples. NumtaDB enlarged the available data by roughly an order of magnitude and, equally importantly, did so by aggregating genuinely heterogeneous sources rather than by expanding a single controlled collection.

The corpus was constructed by combining six independently gathered contributory datasets, conventionally identified by the labels *a* through *f*. Each contributing source was collected by a different group, under a different protocol, from a different population of writers, and with different acquisition equipment. Some sources consist of scanned forms on which writers filled prescribed grids; others consist of photographed handwriting; still others were captured on digitising devices and rendered to raster images. The aggregate corpus in its released form contains approximately eighty-five thousand images, partitioned by its creators into designated training and testing portions.

This assembled character is the defining property of NumtaDB and the source of both its value and its difficulty. Its value lies in the fact that a model trained on it must cope with genuine acquisition diversity rather than the artificially uniform conditions of a single-protocol corpus, which makes performance on NumtaDB a better proxy for real-world performance than performance on a cleaner but narrower dataset would be. Its difficulty lies in the corresponding heterogeneity of image resolution, background colour and texture, contrast polarity, marginal padding, noise characteristics, and label reliability. The creators of the dataset were explicit that the corpus contains augmented samples, samples of varying quality, and a degree of labelling noise [1]. Any study using NumtaDB must therefore make its preprocessing decisions explicit, because those decisions materially affect what is being measured.

3.1.1 Dataset Source and Availability

NumtaDB was released publicly, accompanied by a descriptive technical report, and was distributed through an open competition platform which established it rapidly as the reference benchmark for Bangla numeral recognition [1]. Its images are supplied as raster files with accompanying comma-separated metadata files that record, for each image, its filename, its ground-truth digit label, and provenance information identifying the contributory source from which it originated. The dataset is used in this thesis in its publicly released form, without modification to its labels, and in accordance with the terms of its release. No new data were collected for this study, and no relabelling was performed; the contribution of this thesis is methodological rather than resource-creating, as stated in Section 1.5.

3.2 Dataset Structure and Digit Classes

The label space of the task is the set of ten Bangla numeral classes corresponding to the decimal values zero through nine. Each image in the corpus is annotated with exactly one such label, making this a single-label ten-way classification problem with no multi-label or hierarchical structure.

Table 3.1 enumerates the ten classes together with the transliterated Bangla names of the digits and a description of the dominant visual characteristics of each glyph. The final column notes, for each digit, the classes with which it is most readily confused. These confusion notes are not speculative: they are stated here on the basis of the structural properties of the glyphs, and Chapter 5 will show that the empirical confusion matrix of the trained model corresponds closely to them, which is one of the principal analytical findings of this thesis.

Table 3.1 The ten Bangla numeral classes, their transliterated names, dominant visual characteristics, and the classes with which each is structurally most confusable. The rightmost column anticipates the empirical confusion analysis of Section 5.3.

Label	Bangla name	Dominant visual characteristics	Confusable with
0	*shunno*	Closed oval or near-circular loop; minimal internal structure	4, 5
1	*ek*	Vertical dominant stroke with an upper hook or flag; open form	2, 9
2	*dui*	Curved upper arc descending into a lower horizontal terminal	1, 3
3	*tin*	Double-curved form; upper and lower bowl-like segments	6, 2
4	*char*	Angular junction with an enclosed or partially enclosed region	0, 5
5	*panch*	Curved upper element over an angular lower stroke	6, 3
6	*chhoy*	Prominent loop with an ascending or descending tail	3, 5
7	*shat*	Angular composite of a hook and an oblique stroke	2, 8
8	*aat*	Compound form with a distinctive crossing or junction	3, 2
9	*noy*	Loop element attached to a strong vertical or oblique stroke	1, 8

Two observations about Table 3.1 inform the remainder of this thesis. First, the confusable pairs are, in almost every case, *reciprocal but asymmetric*: digit one is confusable with nine and nine with one, but the frequency of the two confusions need not be equal, because the secondary feature that distinguishes them may be more often omitted in one direction than the other. Section 5.3 shows exactly this asymmetry in the empirical results, with 25 instances of nine misread as one against 19 of one misread as nine. Second, the

confusions cluster around a small number of structural motifs, principally the presence or absence of a closed loop, the curvature of a terminal stroke, and the relative prominence of a vertical component. This clustering is what makes the error analysis of Chapter 5 interpretable rather than merely descriptive.

3.3 Duplicate Removal

The single most consequential data preparation decision taken in this study concerns duplicate images, and it is treated here at length because it is routinely omitted from reports in this literature.

3.3.1 Why Duplicates Matter

An assembled corpus aggregates independently collected sources. Where two contributing sources drew on overlapping populations, employed overlapping collection instruments, or applied augmentation procedures that generated identical outputs from identical inputs, the aggregate corpus may contain images that are byte-for-byte or pixel-for-pixel identical. Such duplicates are harmless in themselves, but they become a serious methodological problem at the moment the corpus is partitioned.

Consider an image $\mathbf{x}$ that appears twice in the corpus, once assigned to the training partition and once to the test partition. During training the model observes $\mathbf{x}$ and adjusts its parameters to classify it correctly. At evaluation the model is presented with the same image and, unsurprisingly, classifies it correctly. This correct prediction is recorded as evidence of generalisation, but it is nothing of the kind: it is memorisation. Each such duplicate inflates the reported test accuracy by an amount that depends on how difficult the image would have been had the model not already seen it, and the inflation is systematically largest for precisely the hardest images, since those are the ones the model would otherwise have been most likely to misclassify.

The magnitude of this effect scales with the duplicate rate. It is therefore not sufficient to observe that duplicates are few; the correct procedure is to remove them before partitioning, so that the training and evaluation sets are disjoint by construction, and to report the number removed so that readers can assess the scale of the issue.

3.3.2 Procedure and Outcome

Duplicate detection in this study targeted *exact* duplicates, that is, images whose pixel arrays are identical after decoding. The procedure operated on the preprocessed representation, so that images differing only in their stored file format but identical in pixel content were correctly identified as duplicates. Each image was reduced to a canonical fixed-length representation and compared against all previously observed images; where a match was found, the later occurrence was discarded and the earlier retained, together with its label.

Applying this procedure to the corpus identified and removed **18 exact duplicate images**. The resulting working corpus, used for all experiments reported in this thesis, contains **72,027 unique handwritten digit images**.

Two remarks are warranted. First, 18 duplicates in a corpus of this size is a low rate, and the accuracy inflation that would have resulted from leaving them in place would have been correspondingly small. The point of performing and reporting the step is not that the effect was large in this instance but that its magnitude was *unknown until measured*, and that a study which does not perform the check cannot state that its evaluation is

uncontaminated. Second, the procedure targeted exact duplicates only. Near-duplicates, images that differ by a small geometric transformation, a slight change in intensity, or minor noise, are not detected by exact matching and may remain in the corpus. Section 3.6 discusses this residual limitation, and Section 6.4 records it among the limitations of the study.

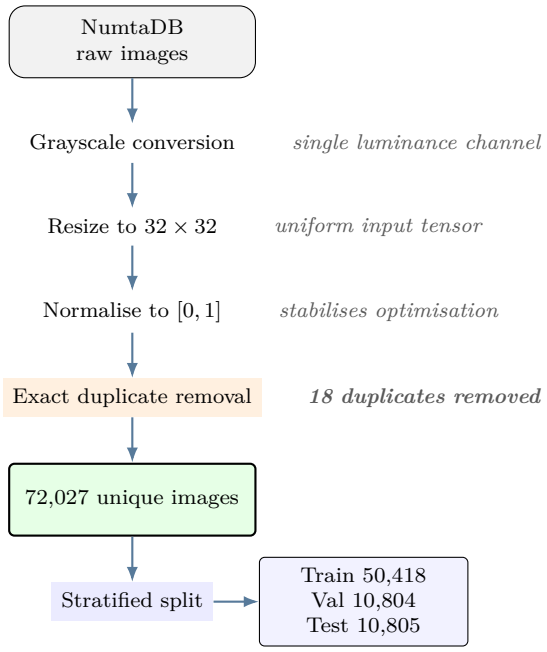

Fig. 3.1 Data preparation workflow. Duplicate removal is performed *before* partitioning, so that the training, validation and test subsets are disjoint by construction.

3.4 Preprocessing

Preprocessing converts the heterogeneous raw images of the assembled corpus into a uniform tensor representation suitable for convolutional processing. Four operations were applied, in the order given.

Grayscale conversion. Source images arriving in three-channel colour form were converted to a single luminance channel. Colour carries no class-discriminative information in this task: the identity of a handwritten digit is determined entirely by the spatial configuration of ink, and the colour of the ink or the paper is an artefact of the acquisition source. In fact, retaining colour would be actively harmful, because the contributory sources of NumtaDB differ systematically in colour characteristics, and a model with access to colour could exploit those differences as a spurious shortcut, learning to identify the *source* rather than the *digit*. Discarding colour removes this shortcut and additionally reduces the input tensor by a factor of three.

Resizing to 32×32 pixels. All images were resampled to a uniform spatial resolution of 32×32 pixels. Uniformity is a requirement of batched convolutional processing. The particular choice of 32 reflects a considered trade-off. A lower resolution such as 16×16 would risk destroying the fine structural details, loop closures and terminal stroke curvature, that distinguish the confusable digit pairs identified in Table 3.1. A higher resolution

such as 64×64 would preserve more detail but would quadruple the spatial cost of every convolutional layer, which conflicts directly with the efficiency objective. At 32×32 the digit occupies enough pixels for loop closure to remain discernible, while the four-stage downsampling of the proposed architecture reduces the map cleanly through 16×16, 8×8 and 4×4 without awkward remainders. The choice of a power of two is not incidental; it makes each stride-two pooling operation exact.

Intensity normalisation. Pixel intensities, originally integers in $[0, 255]$, were rescaled to real values in $[0, 1]$ by division by 255:

$$\tilde{x}_{ij} = \frac{x_{ij}}{255}, \qquad \tilde{x}_{ij} \in [0, 1]. \tag{3.1}$$

Normalisation to a small bounded range is standard practice and is important for optimisation stability. Inputs on the scale of hundreds produce large activations in the first layer, which in turn produce large gradients, which interact poorly with the adaptive step sizes of the Adam optimiser and can cause the divergent behaviour visible in early training. Rescaling to the unit interval places all inputs on a common, modest scale.

Duplicate removal. Applied as described in Section 3.3, removing 18 images and yielding the 72,027-image working corpus.

Table 3.2 summarises the pipeline together with the rationale for each stage.

Table 3.2 Preprocessing operations applied to every image, with the effect on the data representation and the rationale for each.

#	Operation	Effect	Rationale
1	Grayscale conversion	$H \times W \times 3 \rightarrow H \times W \times 1$	Colour is class-irrelevant and correlates with source, creating a spurious shortcut
2	Resize to 32×32	$H \times W \times 1 \rightarrow 32 \times 32 \times 1$	Uniform tensor shape; balances detail retention against computational cost
3	Normalise to $[0, 1]$	$[0, 255] \rightarrow [0, 1]$	Stabilises gradient magnitudes and optimiser behaviour
4	Remove exact duplicates	-18 images	Prevents train–test contamination and accuracy inflation

3.5 Data Augmentation

Data augmentation expands the effective size of the training set by applying label-preserving transformations to training images, so that the model observes a different perturbed variant of each sample on each epoch. Its purpose is regularisation: by forcing the model to classify many transformed versions of the same digit consistently, augmentation encourages representations that are invariant to nuisance variation and discourages memorisation of individual training images. The design space of image augmentation is large, and the taxonomy of **?**] distinguishes geometric transformations, which alter the spatial arrangement of pixels, from photometric transformations, which alter their intensities. The strategy adopted here is drawn entirely from the former category, a decision whose consequences for out-of-distribution performance are examined in Section 5.5.

The augmentation strategy adopted here is purely geometric and comprises four transformations, applied stochastically and in combination.

Random rotation. Small-angle rotation about the image centre models the natural variation in the angle at which writers orient their strokes and at which documents are placed on a scanner bed. It is label-preserving provided the angle remains small; large rotations must be avoided because sufficiently rotated digits can become genuinely ambiguous or can approach the appearance of a different class.

Random width shift. Horizontal translation of the digit within the frame models variation in horizontal centring. Because convolution is translation-equivariant, the network is already partially robust to translation, but the global average pooling stage and the finite receptive field mean that this robustness is not complete, and explicit translation augmentation strengthens it.

Random height shift. Vertical translation, with the same motivation as width shift, modelling variation in baseline placement and vertical centring within the cropped region.

Random zoom. Scaling the digit within the frame models variation in the size at which writers form their digits and in the tightness of the crop produced by an upstream segmentation stage. This is arguably the most valuable of the four for deployment realism, because crop tightness is precisely the property most likely to differ between the curated corpus and a real segmentation pipeline.

Two categories of transformation were deliberately *excluded*. Horizontal and vertical flipping were not applied, because reflection is not label-preserving for numerals: a mirrored digit is generally not a valid digit at all, and where it resembles one it resembles the wrong one. Severe shearing and large-angle rotation were likewise avoided, on the grounds that transformations strong enough to alter the topological relationships within a glyph, for instance by closing an open curve or opening a closed loop, would introduce label noise rather than useful invariance. Given that the confusable pairs of Table 3.1 are distinguished precisely by such topological features, this restraint is essential.

Augmentation was applied only to the training partition. The validation and test partitions were left unaugmented, since their purpose is to estimate performance on the true data distribution rather than on a perturbed variant of it.

Table 3.3 Geometric augmentation transformations applied to the training partition, with the real-world variation each is intended to model.

Transformation	**Models**	**Justification**
Random rotation	Writing-angle and scan-orientation variation	Small angles preserve the label; large angles risk class ambiguity
Random width shift	Horizontal centring variation	Strengthens translation robustness beyond that given by convolution alone
Random height shift	Baseline and vertical placement variation	As above, in the vertical direction
Random zoom	Writer digit size and crop tightness	Most directly models mismatch with a real upstream segmentation stage
Deliberately excluded:		
Horizontal / vertical flip	—	Not label-preserving for numerals
Large rotation / shear	—	Can alter glyph topology and induce label noise

3.6 Dataset Challenges

Five characteristics of the corpus complicate learning and merit explicit statement.

Source heterogeneity. The six contributory datasets differ in resolution, background colour and texture, contrast polarity, marginal padding and noise. Grayscale conversion and resizing normalise the most egregious of these differences, but residual variation persists and constitutes genuine within-class variance that the model must accommodate. This is a burden during training but an asset for deployment realism, since a model trained on homogeneous data would generalise worse to unseen acquisition conditions.

Structural class similarity. As set out in Table 3.1, several digit pairs share dominant strokes. This is a property of the script, not of the dataset, and it establishes a floor below which classification error is unlikely to fall for isolated digits presented without context.

Writing style variation. The corpus aggregates contributions from a large and diverse population of writers, spanning regional and generational differences in letterform conventions. Within-class variance is therefore high, and the classifier must learn decision boundaries robust to stylistic variation that in some cases approaches the magnitude of between-class variation.

Residual near-duplicates. The deduplication of Section 3.3 removes exact matches only. Images differing by a small geometric perturbation or by minor noise remain. Some of these arise from augmentation applied by the dataset creators prior to release [1]. Their presence means that the disjointness guaranteed between partitions is exact-match disjointness, not semantic disjointness, and reported accuracy may retain a small residual optimism.

Label noise. As with any large corpus annotated at scale, a proportion of labels is likely to be incorrect, whether through annotator error or through genuine illegibility of the underlying handwriting. Label noise places a hard ceiling on achievable accuracy: an image whose recorded label is wrong cannot be classified "correctly" by any model that has in fact read it correctly. This consideration is directly relevant to interpreting the 162 residual errors reported in Chapter 5, some fraction of which are likely attributable to annotation rather than to model failure.

3.7 Dataset Statistics and Partitioning

The 72,027 unique images were partitioned into training, validation and test subsets in the ratio $70 : 15 : 15$ using *stratified* sampling, meaning that the sampling was performed independently within each of the ten digit classes so that the class proportions of the full corpus are preserved in each subset. Stratification matters for two reasons. It ensures that no class is under-represented in the validation or test partitions, which would make the per-class metrics of Section 5.4 unreliable for that class; and it ensures that the class prior seen during training matches that seen at evaluation, so that the model is not systematically miscalibrated.

The resulting partition sizes are given in Table 3.4.

The three subsets serve distinct and non-interchangeable roles. The *training* subset is the only data from which parameters are estimated. The *validation* subset is used during

Table 3.4 Stratified partition of the deduplicated corpus into training, validation and test subsets.

Subset	Number of images	Proportion
Training	50,418	70%
Validation	10,804	15%
Testing	10,805	15%
Total	**72,027**	**100%**

training to monitor generalisation, to drive the learning-rate reduction schedule, to trigger early stopping, and to select which epoch's weights are retained; it therefore influences the final model indirectly and cannot be regarded as untouched. The *test* subset is used exactly once, after training is complete, to produce the figures reported in Chapter 5. It plays no role in any training decision, which is what licenses its interpretation as an unbiased estimate of generalisation performance.

The per-class support counts of the test partition, which follow directly from stratified sampling, are reported in Table 5.7 of Chapter 5 and range from 1,067 to 1,088 images per class. The near-uniformity of these counts confirms that the underlying corpus is close to class-balanced and that stratification preserved this balance. This is a convenient property, because it means that accuracy is not distorted by class imbalance and that macro-averaged and weighted-averaged metrics coincide almost exactly, as indeed they do in the results.

Figure 3.2 visualises the partition.

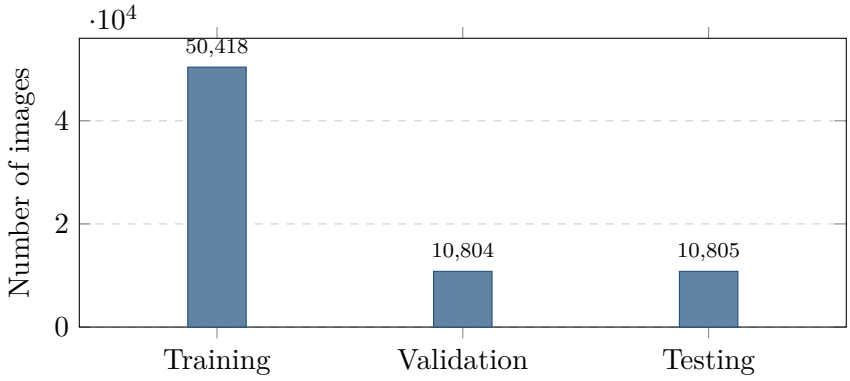

Fig. 3.2 Stratified partition of the 72,027 deduplicated NumtaDB images into training, validation and test subsets in a 70 : 15 : 15 ratio.

3.8 Summary

This chapter has documented the data foundation of the thesis. The NumtaDB corpus was described, together with the assembled and heterogeneous character that gives it both its realism and its difficulty. The ten Bangla numeral classes were enumerated along with the structural similarities that make certain pairs intrinsically confusable, a taxonomy that Chapter 5 will show to be predictive of the trained model's actual errors. The removal of 18 exact duplicate images before partitioning was described and justified at length, yielding a working corpus of 72,027 unique samples. The four-stage preprocessing pipeline and the four-transform geometric augmentation strategy were specified, along with the transformations deliberately excluded and the reasons for excluding them. Five dataset

challenges were stated candidly, including the residual near-duplicate and label-noise issues that remain after deduplication. Finally, the stratified 70 : 15 : 15 partition into 50,418 training, 10,804 validation and 10,805 test images was reported, with the distinct role of each subset made explicit. The architecture that consumes this data is the subject of Chapter 4.

Chapter 4

Proposed Methodology

This chapter presents the proposed method in full. It begins with an overview of the complete workflow, then develops the mathematical foundations of the operations from which the network is built, treating standard convolution, depthwise convolution, pointwise convolution and their separable composition in sequence and deriving the efficiency gain that motivates the design. It then explains each remaining component, batch normalisation, rectified linear activation, pooling, residual connections, global average pooling, dropout and softmax classification, before assembling them into the complete architecture and giving a layer-by-layer parameter accounting. The chapter closes with the training configuration and formal definitions of the evaluation metrics used in Chapter 5.

4.1 Overview of the Methodology

The methodology proceeds in six stages. Raw NumtaDB images are *preprocessed* into a uniform $32 \times 32 \times 1$ normalised tensor representation and deduplicated, as described in Chapter 3. The resulting corpus is *partitioned* by stratified sampling into training, validation and test subsets. Training images are *augmented* stochastically with geometric transformations. The augmented stream is used to *train* the proposed lightweight convolutional network under the Adam optimiser with a callback regime governing learning-rate reduction, early stopping and checkpoint selection. The retained model is then *evaluated* on the untouched test partition. Finally the model is *analysed*, both through aggregate and per-class metrics and through the confusion structure of its residual errors.

Figure 4.1 presents this workflow.

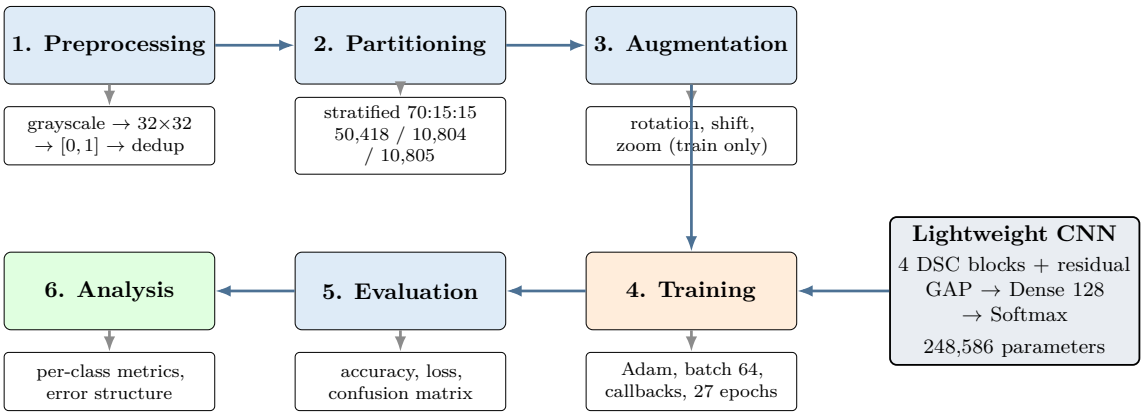

Fig. 4.1 Complete methodology workflow, from raw NumtaDB images through preprocessing, partitioning and augmentation to training, evaluation and error analysis of the proposed lightweight convolutional network.

4.2 Mathematical Foundations of the Convolutional Operations

The efficiency of the proposed architecture rests entirely on the replacement of standard convolution by its depthwise separable factorisation. To make that efficiency claim precise rather than rhetorical, this section derives the parameter and computational cost of each operation from first principles.

4.2.1 Standard Convolution

Let the input to a convolutional layer be a feature map $\mathbf{X} \in \mathbb{R}^{H \times W \times M}$, where H and W are the spatial height and width and M is the number of input channels. Let the layer produce an output $\mathbf{Y} \in \mathbb{R}^{H' \times W' \times N}$ with N output channels. A standard convolution is parameterised by a four-dimensional kernel tensor $\mathbf{K} \in \mathbb{R}^{K \times K \times M \times N}$, where K is the spatial kernel extent, taken as $K = 3$ throughout this work.

With unit stride and appropriate padding so that $H' = H$ and $W' = W$, the output at spatial location (i, j) in output channel n is

$$Y_{i,j,n} \;=\; \sum_{m=1}^{M} \sum_{u=1}^{K} \sum_{v=1}^{K} K_{u,v,m,n} \cdot X_{i+u-\lceil K/2 \rceil,\, j+v-\lceil K/2 \rceil,\, m}. \tag{4.1}$$

Equation (4.1) exposes the essential structure of the operation: the triple summation performs *two distinct functions simultaneously*. The summations over u and v perform *spatial* filtering, combining neighbouring pixels to detect local patterns. The summation over m performs *cross-channel* combination, mixing the responses of different input channels into each output channel. It is the simultaneity of these two functions that makes standard convolution expensive, and their separability that makes the depthwise factorisation possible.

The parameter count of the layer, ignoring bias terms which are subsumed into the following batch normalisation, is

$$P_{\text{std}} \;=\; K^2 \cdot M \cdot N, \tag{4.2}$$

and the number of multiply–accumulate operations required to produce the full output map is

$$C_{\text{std}} \;=\; K^2 \cdot M \cdot N \cdot H' \cdot W'. \tag{4.3}$$

Both quantities are proportional to the *product* $M \cdot N$, which is the source of the cost: as a network deepens and its channel count grows, the cost of each layer grows quadratically in channel width.

4.2.2 Depthwise Convolution

Depthwise convolution performs spatial filtering only, and does so independently within each input channel. Its kernel is $\mathbf{K}^{\text{dw}} \in \mathbb{R}^{K \times K \times M}$, comprising exactly one $K \times K$ spatial filter per input channel, and its output has the same channel count M as its input:

$$\hat{Y}_{i,j,m} \;=\; \sum_{u=1}^{K} \sum_{v=1}^{K} K^{\text{dw}}_{u,v,m} \cdot X_{i+u-\lceil K/2 \rceil,\, j+v-\lceil K/2 \rceil,\, m}. \tag{4.4}$$

The summation over m present in (4.1) is absent from (4.4): channel m of the output

depends only on channel m of the input. There is no cross-channel mixing whatsoever. The parameter count and computational cost are correspondingly

$$P_{\text{dw}} = K^2 \cdot M, \qquad C_{\text{dw}} = K^2 \cdot M \cdot H' \cdot W'. \tag{4.5}$$

Both are now *linear* in M rather than proportional to $M \cdot N$. The saving is dramatic, but so is the loss of function: a network of depthwise convolutions alone could never combine information across channels, and would therefore be unable to build composite features from simpler ones. The pointwise stage restores this capability.

4.2.3 Pointwise Convolution

Pointwise convolution is a convolution with $K = 1$. Its kernel is $\mathbf{K}^{\text{pw}} \in \mathbb{R}^{1\times1\times M\times N}$ and it computes

$$Y_{i,j,n} \;=\; \sum_{m=1}^{M} K^{\text{pw}}_{1,1,m,n} \cdot \hat{Y}_{i,j,m}. \tag{4.6}$$

Because the spatial extent is unity, the operation examines exactly one spatial position at a time and performs no spatial filtering at all. What it does perform is a full linear recombination across channels: it is, at each spatial location independently, a learned $M \to N$ linear map. Its cost is

$$P_{\text{pw}} = M \cdot N, \qquad C_{\text{pw}} = M \cdot N \cdot H' \cdot W'. \tag{4.7}$$

This retains the $M{\cdot}N$ product but eliminates the K^2 factor. The pointwise stage additionally performs the channel-count change: it is here, and only here, that the number of channels is altered from M to N.

4.2.4 Depthwise Separable Convolution

The depthwise separable convolution is the composition of the two preceding operations: a depthwise convolution that filters spatially within channels, followed by a pointwise convolution that recombines channels. Its total cost is the sum of (4.5) and (4.7):

$$P_{\text{dsc}} = K^2M + MN, \tag{4.8}$$

$$C_{\text{dsc}} = (K^2M + MN) \cdot H' \cdot W'. \tag{4.9}$$

The ratio of separable to standard cost is therefore

$$\rho \;=\; \frac{P_{\text{dsc}}}{P_{\text{std}}} \;=\; \frac{K^2M + MN}{K^2MN} \;=\; \frac{1}{N} + \frac{1}{K^2}. \tag{4.10}$$

Equation (4.10) is the central result of this section, and several features of it deserve comment. The ratio is *independent of the input channel count* M, which cancels entirely. It depends only on the output channel count N and the kernel size K. For the 3×3 kernels used throughout this work, $1/K^2 = 1/9 \approx 0.1111$, and as N grows the $1/N$ term vanishes, so ρ approaches $1/9$ asymptotically. This means the factorisation can never save more than a factor of nine with 3×3 kernels, and that it approaches that bound most closely in the wide, deep layers where the absolute cost is largest, which is precisely where saving matters most.

Table 4.1 evaluates (4.10) at the four output channel widths used in the proposed

architecture.

Table 4.1 Theoretical cost ratio of depthwise separable to standard convolution, from Equation (4.10), evaluated at $K = 3$ for the four channel widths of the proposed architecture. The reduction factor is $1/\rho$.

Output channels N	$1/N$	$\rho = 1/N + 1/9$	**Reduction** $1/\rho$
32	0.03125	0.14236	7.02×
64	0.01563	0.12674	7.89×
128	0.00781	0.11892	8.41×
256	0.00391	0.11502	8.69×
∞	0	0.11111	9.00× (bound)

To make the practical consequence concrete, consider the convolutional weight cost of the proposed topology computed both ways. Under standard convolution with 3×3 kernels, the four blocks would require $9 \cdot 1 \cdot 32 = 288$, $9 \cdot 32 \cdot 64 = 18{,}432$, $9 \cdot 64 \cdot 128 = 73{,}728$ and $9 \cdot 128 \cdot 256 = 294{,}912$ weights respectively, totalling 387,360. Under the separable factorisation the same four blocks require $9 \cdot 1 + 1 \cdot 32 = 41$, $9 \cdot 32 + 32 \cdot 64 = 2{,}336$, $9 \cdot 64 + 64 \cdot 128 = 8{,}768$ and $9 \cdot 128 + 128 \cdot 256 = 33{,}920$ weights, totalling 45,065. The factorisation therefore reduces the pure convolutional weight cost of the feature extractor by a factor of approximately 8.6×. This is the single largest source of the architecture's compactness, and it answers RQ2 directly.

4.3 Component Layers

4.3.1 Batch Normalisation

Batch normalisation standardises the distribution of each channel's activations using statistics computed over the current mini-batch, then applies a learned affine transformation [**?**]. For a mini-batch $\mathcal{B} = \{z^{(1)}, \ldots, z^{(B)}\}$ of pre-activation values in a given channel, the operation computes

$$\mu_{\mathcal{B}} = \frac{1}{B} \sum_{b=1}^{B} z^{(b)}, \tag{4.11}$$

$$\sigma_{\mathcal{B}}^2 = \frac{1}{B} \sum_{b=1}^{B} \big(z^{(b)} - \mu_{\mathcal{B}}\big)^2, \tag{4.12}$$

$$\hat{z}^{(b)} = \frac{z^{(b)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \tag{4.13}$$

$$\tilde{z}^{(b)} = \gamma\, \hat{z}^{(b)} + \beta, \tag{4.14}$$

where ε is a small constant preventing division by zero and γ, β are learned per-channel scale and shift parameters. The affine stage of (4.14) is essential: without it, normalisation would forcibly constrain every channel to zero mean and unit variance, removing the network's ability to choose a different distribution where one is useful. With it, the network can recover the identity transformation if that is optimal.

Batch normalisation contributes four parameters per channel, of which two, γ and β, are trainable, and two, the running estimates of mean and variance used at inference time, are non-trainable buffers updated by exponential moving average during training. This distinction accounts directly for the 3,968 non-trainable parameters reported for the proposed model in Section 4.5.

Three benefits justify its use here. It permits substantially higher learning rates by preventing the activation distribution of any layer from drifting as the layers below it update, which is what makes the initial learning rate of 10^{-3} viable; later analysis attributes this benefit primarily to a smoothing of the loss landscape rather than to the reduction of covariate shift originally proposed [**?**]. It provides mild regularisation, because the normalisation of each sample depends on the composition of its mini-batch, injecting noise that discourages memorisation. And it reduces sensitivity to weight initialisation, which would otherwise require the variance-preserving schemes of **?**] and **?**] to be applied with care. It also has a cost relevant to Section 5.1: because training uses batch statistics while inference uses running estimates, a mismatch between the two produces a discrepancy between training and validation metrics that can be severe early in training, before the running estimates have converged. This mechanism is the primary explanation for the validation instability observed in the recorded training history.

4.3.2 Rectified Linear Activation

The rectified linear unit is defined elementwise as

$$\mathrm{ReLU}(z) = \max(0, z), \tag{4.15}$$

with derivative

$$\frac{\partial\,\mathrm{ReLU}(z)}{\partial z} = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z < 0, \end{cases} \tag{4.16}$$

undefined at the origin and conventionally taken as zero there [**?**].

ReLU's advantages over the saturating sigmoid and hyperbolic tangent activations it displaced are threefold. Its gradient is exactly unity for all positive inputs, so gradients propagate backwards through many layers without the exponential attenuation that saturating functions produce, which is the classical vanishing-gradient problem. It is computationally trivial, requiring a single comparison. And it induces sparsity, since all negative pre-activations map to exactly zero, yielding representations in which a substantial fraction of units are inactive for any given input, which tends to produce more disentangled features. Its known weakness is the "dying ReLU" phenomenon, in which a unit whose pre-activation is negative for every training input receives zero gradient permanently and ceases to learn. Batch normalisation immediately preceding each activation substantially mitigates this by centring the pre-activation distribution, which is one reason the two operations are almost always paired, as they are throughout the proposed architecture.

4.3.3 Max Pooling

Max pooling reduces spatial resolution by partitioning each channel's feature map into non-overlapping windows and retaining only the maximum value within each. For a 2×2 window with stride 2,

$$Y_{i,j,m} = \max_{0 \le u,v \le 1} X_{2i+u,\,2j+v,\,m}, \tag{4.17}$$

which halves both spatial dimensions and therefore quarters the number of spatial positions, leaving the channel count unchanged. The operation has no learnable parameters.

Pooling serves three purposes. It reduces computation in all subsequent layers, since every later operation acts on a quarter as many positions. It enlarges the effective receptive field of subsequent layers relative to the original input, so that after several pooling stages a 3×3 kernel covers a large fraction of the input image, allowing the network to reason about whole-glyph structure. And it confers a degree of local translation invariance, since the maximum within a window is unchanged by small displacements of the feature that produced it. The choice of maximum rather than average is appropriate for handwriting because the salient evidence is the *presence* of a stroke feature: a maximum records that a feature was detected somewhere in the window, whereas an average would dilute a strong localised response by surrounding empty background.

The proposed architecture applies 2×2 max pooling with stride 2 within each of its four blocks, and the effective spatial reduction across the network is documented in Table 4.2.

4.3.4 Residual Connections

A residual block computes

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \boldsymbol{\theta}) + \mathcal{S}(\mathbf{x}), \tag{4.18}$$

where $\mathcal{F}$ is the block's learned transformation, here the depthwise separable convolution with its normalisation and activation, and $\mathcal{S}$ is a shortcut mapping which is the identity when the input and output shapes agree and a projection otherwise [14].

The value of this formulation is most clearly seen in the backward pass. Differentiating (4.18) with respect to the input gives

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}} + \mathbf{I}, \tag{4.19}$$

where the additive identity term guarantees that the gradient reaching $\mathbf{x}$ can never be smaller than the gradient arriving at $\mathbf{y}$ by more than the learned term permits. Gradient signal therefore has an unobstructed path through the network regardless of how the learned branches behave, which is what eliminates the degradation phenomenon that had previously limited depth.

In a shallow four-block network the vanishing-gradient motivation is weaker than in a hundred-layer network, and it would be misleading to claim otherwise. Two other justifications apply more directly here. First, the skip path improves the conditioning of the optimisation landscape even at modest depth, which in combination with batch normalisation is what makes stable training at a 10^{-3} learning rate possible on augmented data. Second, and more important for a capacity-constrained model, the skip connection reformulates each block's task from *constructing* a representation to *refining* the one it receives. Because the identity is available for free, none of the block's limited parameter budget need be spent learning to preserve information that is already present; the entire budget can be devoted to the incremental transformation. In a network whose total capacity is deliberately restricted to under three hundred thousand parameters, this efficiency of parameter use is a substantive benefit.

Where a block changes the channel count, as every block in the proposed architecture does, the identity shortcut is dimensionally inadmissible and $\mathcal{S}$ must be a projection, conventionally a 1×1 convolution with matching stride, which contributes a small number of additional parameters.

4.3.5 Global Average Pooling

Global average pooling reduces each channel of a feature map to a single scalar by averaging over all spatial positions [**?**]. For an input $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ it produces $\mathbf{g} \in \mathbb{R}^{C}$ with

$$g_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} X_{i,j,c}. \tag{4.20}$$

This operation is, for a compact architecture, arguably the single most valuable design choice available, and its importance is best appreciated by comparison with the alternative it replaces. The conventional approach is to *flatten* the final feature map into a vector and feed it to a fully connected layer. In the proposed architecture the final feature map is $4 \times 4 \times 256$, so flattening would yield a vector of $4 \cdot 4 \cdot 256 = 4{,}096$ elements. Connecting this to a 128-unit dense layer would require $4{,}096 \times 128 = 524{,}288$ weights in that single layer alone, which is more than twice the parameter count of the *entire* proposed model. Global average pooling instead reduces the map to a 256-element vector, and the corresponding dense layer requires $256 \times 128 = 32{,}768$ weights, a sixteenfold reduction.

Beyond parameter economy, global average pooling has two further benefits. It acts as a strong structural regulariser: because it has no parameters and enforces a direct correspondence between each channel and a single scalar feature, it cannot overfit, and it pushes the network towards learning channels that are individually meaningful class evidence rather than relying on an arbitrary dense recombination of spatial positions. And it makes the network agnostic to input spatial size, since the averaging accommodates any $H \times W$, which is a useful property for future deployment on inputs of differing resolution.

4.3.6 Dropout

Dropout regularises a network by randomly zeroing a fraction of activations during training [**?**]. With retention probability p, each unit is independently kept with probability p and zeroed otherwise, and the surviving activations are rescaled by $1/p$ so that the expected value of the layer's output is preserved between training and inference:

$$r_k \sim \text{Bernoulli}(p), \qquad \tilde{h}_k = \frac{r_k \, h_k}{p}. \tag{4.21}$$

At inference dropout is disabled and all units are retained.

The mechanism can be understood in two complementary ways. It prevents co-adaptation: a unit cannot rely on the presence of any particular other unit, since that unit may be absent, so each must learn a feature useful in its own right. And it approximates an ensemble: training with dropout is approximately equivalent to training an exponentially large collection of thinned subnetworks with shared weights, and inference with all units retained approximates averaging their predictions.

The proposed architecture applies dropout with a rate of 0.30, meaning 30% of activations are zeroed, at a single location: after the 128-unit dense layer. This placement is deliberate. The dense layer is the point of highest parameter density outside the convolutional stack and therefore the point of greatest overfitting risk. The convolutional layers require less explicit regularisation because weight sharing already constrains them heavily, and because batch normalisation and the geometric augmentation of Section 3.5 provide regularisation of their own. Applying aggressive dropout within the convolutional stack of an already capacity-constrained network would risk underfitting.

4.3.7 Softmax Classification

The final layer maps a 128-dimensional hidden representation to ten class logits $\mathbf{z} \in \mathbb{R}^{10}$, which the softmax function converts to a probability distribution:

$$\big[\mathrm{softmax}(\mathbf{z})\big]_k = \frac{\exp(z_k)}{\sum_{l=0}^{9} \exp(z_l)}, \qquad k = 0, \ldots, 9. \tag{4.22}$$

The output is non-negative and sums to unity by construction, so it is a valid distribution over the ten classes, and the predicted label is its argmax. In practice the exponentials are computed after subtracting $\max_l z_l$ from every logit, which leaves the result unchanged but prevents overflow.

The corresponding loss is categorical cross-entropy. For a single sample with true class y,

$$\mathcal{L}(\boldsymbol{\theta}) = -\log\big[\mathrm{softmax}(\mathbf{z})\big]_y = -z_y + \log \sum_{l=0}^{9} \exp(z_l), \tag{4.23}$$

and over a mini-batch the mean of (4.23) is minimised. The gradient of this loss with respect to the logits takes the notably simple form

$$\frac{\partial \mathcal{L}}{\partial z_k} = \big[\mathrm{softmax}(\mathbf{z})\big]_k - \mathbb{1}[k = y], \tag{4.24}$$

that is, the predicted probability minus the one-hot target. This clean form is one reason the softmax and cross-entropy pairing is universal for multi-class classification: the gradient is bounded, well-scaled, and largest precisely when the prediction is most wrong.

This work uses the *sparse* categorical cross-entropy formulation, which is mathematically identical to (4.23) but accepts integer class labels directly rather than requiring one-hot encoded targets, avoiding the construction of a $50{,}418 \times 10$ target matrix.

4.4 The Proposed Architecture

The complete network is assembled from the components above as follows. An input tensor of shape $32 \times 32 \times 1$ passes through four depthwise separable convolution blocks with output channel widths 32, 64, 128 and 256. Each block comprises a 3×3 depthwise convolution with unit stride and same padding, batch normalisation, rectified linear activation, a 1×1 pointwise convolution, batch normalisation, and 2×2 max pooling with stride 2, together with an additive residual connection spanning the block. The first three blocks apply an effective stride of 2, reducing the spatial resolution from 32×32 to 16×16, then 8×8, then 4×4. The fourth block applies an effective stride of 1, preserving the 4×4 resolution while increasing channel depth to 256.

The resulting $4 \times 4 \times 256$ feature map is reduced by global average pooling to a 256-element vector. This passes to a 128-unit dense layer with batch normalisation and rectified linear activation, followed by dropout at rate 0.30, and finally to a ten-unit dense layer with softmax activation producing the class distribution.

The progressive widening of channels alongside progressive spatial reduction follows the standard convolutional design principle: early layers operate at high spatial resolution on few channels, detecting simple local features such as oriented stroke fragments, while deeper layers operate at low spatial resolution on many channels, detecting complex configurations such as loop closures and stroke junctions. The total quantity of information carried at each stage remains roughly balanced, since each halving of spatial dimensions is accompanied

by a doubling of channel count.

Figure 4.2 presents the architecture diagrammatically, and Table 4.2 gives the layer-by-layer specification.

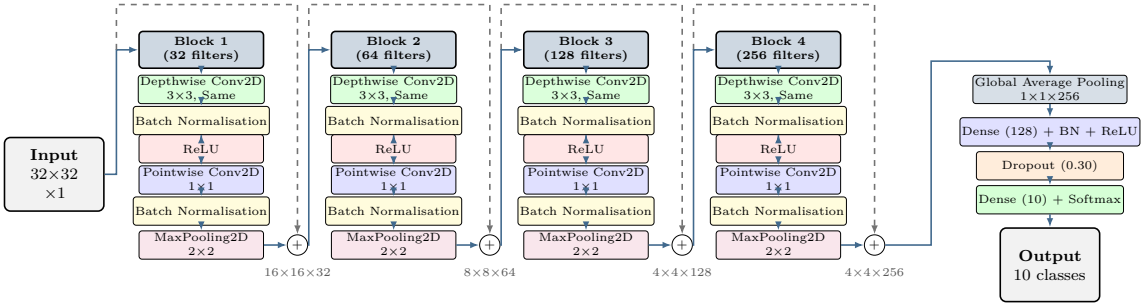

Fig. 4.2 The proposed lightweight convolutional architecture. Four depthwise separable convolution blocks with additive residual connections (dashed) feed a classification head based on global average pooling. Solid arrows carry the main signal path; dashed arrows carry the residual shortcuts.

4.5 Layer Specification and Parameter Accounting

Table 4.2 gives the layer-by-layer specification of the proposed model as reported by the model summary, including output shapes, filter counts, kernel sizes, strides, padding, activations and parameter counts.

Table 4.2 Layer-by-layer specification of the proposed lightweight CNN. DW denotes depthwise convolution and PW pointwise convolution. Parameter counts are as reported by the model summary.

Layer (type)	Output shape	Filters	Kernel size	Stride	Activation	Params
Input	32×32×1	—	—	—	—	0
DS Conv Block 1	16×16×32	32	3×3 DW + 1×1 PW	2	ReLU	2,496
DS Conv Block 2	8×8×64	64	3×3 DW + 1×1 PW	2	ReLU	8,736
DS Conv Block 3	4×4×128	128	3×3 DW + 1×1 PW	2	ReLU	33,024
DS Conv Block 4	4×4×256	256	3×3 DW + 1×1 PW	1	ReLU	131,328
Global Avg. Pooling	1×1×256	—	—	—	—	0
Dense + BN + Dropout	1×1×128	128	—	—	ReLU	32,896
Dense (output)	1×1×10	10	—	—	Softmax	1,290
Total parameters						**248,586**
Trainable parameters						244,618
Non-trainable parameters						3,968

Several observations follow from Table 4.2.

The parameter distribution is heavily skewed towards the deepest convolutional block. Block 4 alone accounts for 131,328 parameters, which is more than the first three blocks combined and more than half of the entire model. This is the expected consequence of the pointwise stage's $M \cdot N$ cost: at $M = 128$ and $N = 256$ the pointwise convolution

requires $128 \times 256 = 32{,}768$ weights, and the block's associated projection and normalisation parameters add further. This concentration identifies block 4 as the natural first target for any future compression effort, a point taken up in Section 6.5.

The classification head is remarkably cheap. The dense layers together account for $32{,}896+1{,}290 = 34{,}186$ parameters, roughly 14% of the model. As computed in Section 4.3.5, a flatten-based head would have required over half a million parameters in a single layer. The global average pooling decision is therefore responsible for the largest single saving available in the architecture, larger even than the depthwise factorisation of any individual block.

The non-trainable count of 3,968 corresponds exactly to the running mean and variance buffers of the batch normalisation layers, at two non-trainable values per normalised channel, and confirms that batch normalisation is applied at every stage described in Section 4.4.

The total of 248,586 parameters corresponds to a stored model of approximately $248{,}586 \times 4 \approx 994$ kilobytes in single-precision floating point, that is, slightly under one megabyte. Under 8-bit post-training quantisation this would fall to roughly 249 kilobytes, though quantisation was not performed in this work and is identified as future work.

4.6 Training Configuration

Table 4.3 lists the complete training configuration.

Table 4.3 Training hyperparameters and configuration.

Parameter	**Value**
Optimiser	Adam
Initial learning rate	0.001
Batch size	64
Maximum epochs	50
Epochs actually trained	27 (terminated by early stopping)
Loss function	Sparse categorical cross-entropy
Activation functions	ReLU (hidden), Softmax (output)
Input size	$32 \times 32 \times 1$
Dropout rate	0.30
Callbacks	EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
EarlyStopping patience	8 epochs
ReduceLROnPlateau patience	3 epochs

Adam optimiser. Adam maintains per-parameter exponential moving averages of the first and second moments of the gradient and uses their ratio to set an adaptive per-parameter step size [**?**]. With g_t the gradient at step t,

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{4.25}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{4.26}$$

and the update is

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \tag{4.27}$$

where the bias corrections of (4.26) compensate for the zero-initialisation of the moment estimates. Adam was chosen for its robustness to hyperparameter choice and its rapid initial progress, which is visible in the recorded history as a jump from 79.36% to 94.53% training accuracy between the first and second epochs.

Batch size. A batch size of 64 balances three considerations. It is large enough for the batch statistics of (4.11) and (4.12) to be reasonably stable estimates, which matters because unstable batch statistics are a direct cause of the validation oscillation analysed in Section 5.1. It is small enough to retain useful gradient noise, which aids escape from sharp minima of the kind that **?**] associate with degraded generalisation under large-batch training. And it fits comfortably in the memory of modest hardware. With 50,418 training images, each epoch comprises $\lceil 50418/64 \rceil = 788$ optimiser steps.

Callback regime. Three callbacks jointly govern training and together resolve the epoch-selection problem raised in Section 1.4.

ReduceLROnPlateau monitors validation loss and multiplicatively reduces the learning rate when no improvement is observed for 3 consecutive epochs. The recorded history shows this mechanism operating five times, producing the schedule $10^{-3} \to 5 \times 10^{-4} \to 2.5 \times 10^{-4} \to 1.25 \times 10^{-4} \to 6.25 \times 10^{-5}$, each step a halving. The effect on stability is examined in Section 5.1.

EarlyStopping terminates training when validation loss has failed to improve for 8 consecutive epochs, preventing both wasted computation and progressive overfitting. The choice of patience is a genuine trade-off of the kind analysed by **?**]: too short a patience halts a run that would still have improved, while too long a patience wastes computation and risks overfitting. A patience of 8 is generous, and deliberately so, given the epoch-to-epoch validation variance documented in Section 5.1. Training stopped after 27 of the permitted 50 epochs.

ModelCheckpoint saves the model weights whenever validation loss reaches a new minimum, and the best such checkpoint is restored at the end of training. This is the decisive callback for a training run exhibiting validation oscillation: rather than retaining whatever weights happen to exist when training halts, the protocol retains the weights that achieved the best validation loss over the entire run. The recorded history shows the minimum validation loss of 0.0525 occurring at epoch 19, and it is precisely this checkpoint that was evaluated on the test set, which explains why the reported test loss equals 0.0525 exactly.

4.7 Evaluation Metrics

Let the confusion matrix be $\mathbf{C} \in \mathbb{Z}^{10\times 10}$, where C_{ij} counts test samples whose true class is i and predicted class is j. For class k define

$$\mathrm{TP}_k = C_{kk}, \quad \mathrm{FP}_k = \sum_{i\neq k} C_{ik}, \quad \mathrm{FN}_k = \sum_{j\neq k} C_{kj}, \quad \mathrm{TN}_k = \sum_{i\neq k}\sum_{j\neq k} C_{ij}. \tag{4.28}$$

Accuracy. The proportion of all samples classified correctly, equal to the trace of the confusion matrix divided by its total:

$$\mathrm{Accuracy} = \frac{\sum_k C_{kk}}{\sum_i \sum_j C_{ij}}. \tag{4.29}$$

Precision. For class k, the proportion of samples predicted as k that truly are k:

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k} = \frac{C_{kk}}{\sum_i C_{ik}}, \tag{4.30}$$

that is, the diagonal entry divided by its *column* sum. Low precision indicates that the class acts as an over-attractive prediction, absorbing samples belonging to others.

Recall. For class k, the proportion of samples truly of class k that are predicted as k:

$$\text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k} = \frac{C_{kk}}{\sum_j C_{kj}}, \tag{4.31}$$

the diagonal entry divided by its *row* sum. Low recall indicates that the class is frequently missed, its samples being assigned elsewhere.

F1-score. The harmonic mean of precision and recall:

$$\text{F1}_k = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}. \tag{4.32}$$

The harmonic mean is used rather than the arithmetic mean because it is dominated by the smaller of its arguments, so a class cannot obtain a high F1-score by excelling on one metric while failing on the other.

Support. The number of test samples truly belonging to class k, $\text{Support}_k = \sum_j C_{kj}$, that is, the row sum.

Macro and weighted averaging. The macro average treats all classes equally,

$$\text{Macro} = \frac{1}{10} \sum_{k=0}^{9} \text{Metric}_k, \tag{4.33}$$

whereas the weighted average weights each class by its support,

$$\text{Weighted} = \frac{\sum_k \text{Support}_k \cdot \text{Metric}_k}{\sum_k \text{Support}_k}. \tag{4.34}$$

The two differ substantially only when class supports are markedly unequal. Because stratified sampling produced near-uniform supports, ranging only from 1,067 to 1,088, the macro and weighted averages reported in Chapter 5 coincide to four decimal places, which is itself a useful confirmation that the partition is balanced.

4.8 Summary

This chapter has specified the proposed method completely. The mathematics of standard, depthwise, pointwise and depthwise separable convolution were derived, yielding the cost ratio $\rho = 1/N + 1/K^2$ of Equation (4.10) and a demonstrated 8.6× reduction in the convolutional weight cost of the feature extractor. Each component layer was explained together with its role in this specific design, with particular attention to global average pooling, which alone avoids over half a million parameters relative to a flatten-based head.

The complete architecture was assembled and its 248,586 parameters accounted for layer by layer, showing that over half reside in the deepest block. The training configuration was specified, including the callback regime whose ModelCheckpoint component explains why the reported test loss coincides exactly with the best validation loss of 0.0525. Finally the evaluation metrics were defined formally in terms of the confusion matrix. Chapter 5 reports and interprets the results obtained under this methodology.

Chapter 5

Results and Evaluation

This chapter reports and interprets the experimental results obtained with the methodology of Chapter 4. It begins with the training and validation trajectories across all 27 epochs, giving particular attention to the pronounced validation instability observed during the high-learning-rate phase and to the mechanism by which the learning-rate schedule suppressed it. It then reports test performance, presents the full confusion matrix and analyses its structure in detail, examines per-class precision, recall and F1-score, reports a qualitative assessment on photographed handwriting, positions the results against prior work, and discusses their implications. Every quantitative figure reported here is an outcome of the experiments described in the preceding chapters.

5.1 Training and Validation Performance

Training proceeded for 27 epochs before early stopping terminated it, well short of the permitted maximum of 50. Table 5.1 records the complete epoch-by-epoch history of training accuracy, training loss, validation accuracy, validation loss and learning rate.

5.1.1 Training Accuracy and Loss

The training trajectory is exemplary and requires little commentary. Training accuracy rose from 79.36% after the first epoch to 94.53% after the second, a gain of over fifteen percentage points in a single epoch that reflects the rapid initial progress characteristic of the Adam optimiser on a well-conditioned problem. Thereafter the curve entered a phase of steady, monotonic refinement, passing 97% at epoch 7, 98% at epoch 9 and 99% at epoch 24, finishing at 99.17%. Training loss fell correspondingly and almost perfectly monotonically from 0.6219 to 0.0261, a reduction of approximately 96%. The only departures from strict monotonicity are trivial, a rise from 0.0402 to 0.0410 between epochs 14 and 15 and from 0.0360 to 0.0367 between epochs 21 and 22, both well within the stochastic variation expected from mini-batch sampling and augmentation.

The smoothness of this trajectory is itself informative. It indicates that the optimisation problem is well-conditioned, that the learning rate is not so high as to cause divergence in the training objective, and that the architecture's capacity is adequate to fit the training data. Had the model been too small for the task, training accuracy would have plateaued well below 99%; the fact that it did not confirms that the parameter budget of 248,586 is sufficient rather than merely tolerable.

Table 5.1 Training and validation performance of the proposed lightweight CNN across all 27 epochs. The best validation performance, at epoch 19, is shown in bold; it is the checkpoint restored by ModelCheckpoint and evaluated on the test set.

Epoch	Train Acc.	Train Loss	Val. Acc.	Val. Loss	Learning Rate
1	79.36%	0.6219	64.51%	1.4797	0.001000
2	94.53%	0.1749	68.17%	1.2930	0.001000
3	95.72%	0.1376	53.74%	1.8750	0.001000
4	96.28%	0.1208	85.45%	0.4808	0.001000
5	96.73%	0.1050	48.39%	3.4994	0.001000
6	96.94%	0.0993	72.44%	1.1810	0.001000
7	97.24%	0.0877	76.60%	0.8584	0.001000
8	97.98%	0.0688	84.32%	0.5081	0.000500
9	98.03%	0.0655	92.35%	0.2237	0.000500
10	98.12%	0.0602	92.69%	0.2363	0.000500
11	98.18%	0.0594	66.61%	1.7269	0.000500
12	98.19%	0.0574	78.78%	0.7125	0.000500
13	98.28%	0.0553	92.10%	0.2443	0.000500
14	98.63%	0.0450	73.75%	1.1692	0.000250
15	98.70%	0.0402	78.77%	0.8695	0.000250
16	98.75%	0.0410	94.86%	0.1601	0.000250
17	98.72%	0.0416	96.21%	0.1283	0.000250
18	98.79%	0.0400	66.54%	2.0701	0.000250
19	**98.85%**	**0.0380**	**98.39%**	**0.0525**	**0.000250**
20	98.84%	0.0371	70.04%	1.3214	0.000250
21	98.86%	0.0360	62.80%	2.2693	0.000250
22	98.84%	0.0367	76.19%	1.0992	0.000250
23	98.97%	0.0312	97.44%	0.0796	0.000125
24	99.05%	0.0295	98.21%	0.0597	0.000125
25	99.07%	0.0290	96.61%	0.1152	0.000125
26	99.16%	0.0266	97.18%	0.0972	0.0000625
27	99.17%	0.0261	97.85%	0.0687	0.0000625

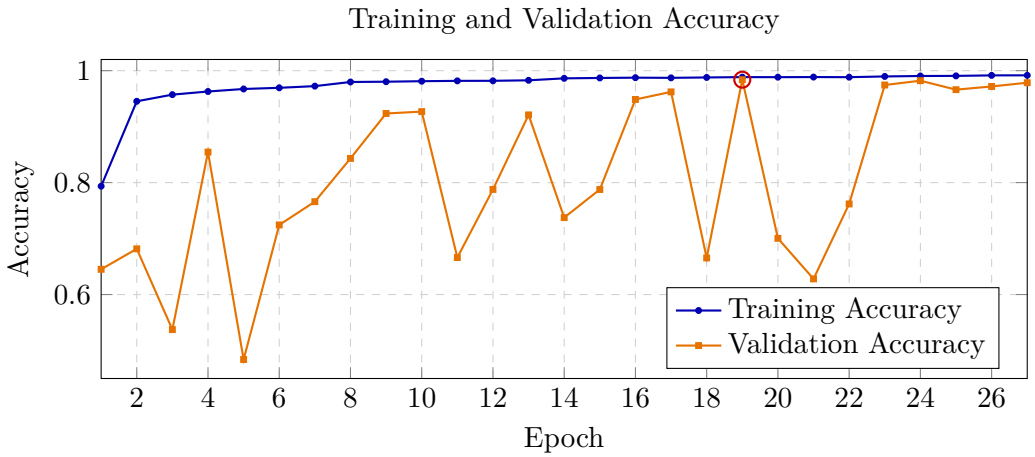

Fig. 5.1 Training and validation accuracy across 27 epochs. The training curve rises smoothly and monotonically; the validation curve oscillates severely during the high-learning-rate phase before stabilising after successive learning-rate reductions. The circled point at epoch 19 marks the checkpoint restored for testing.

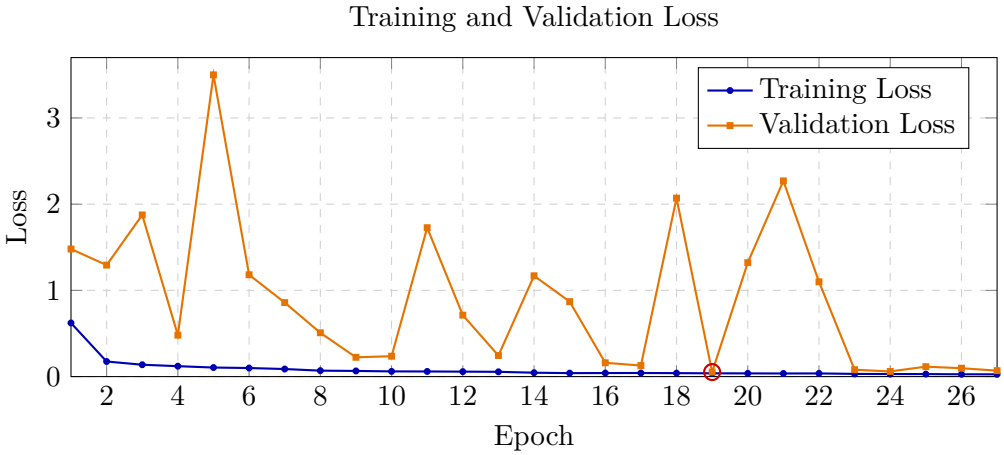

Fig. 5.2 Training and validation loss across 27 epochs. Validation loss peaks at 3.4994 at epoch 5 and oscillates repeatedly before converging; the minimum of 0.0525 occurs at epoch 19 (circled).

5.1.2 Validation Instability and Its Explanation

The validation trajectory shows considerably larger fluctuations than the training trajectory and represents the most notable behaviour in the experimental record.

During the initial training phase with a learning rate of 10^{-3}, validation accuracy fluctuated significantly. It decreased from 64.51% at epoch 1 to 53.74% at epoch 3, increased to 85.45% at epoch 4, and dropped to 48.39% at epoch 5, where validation loss reached 3.4994. These large changes occurred while training accuracy continued to improve smoothly, creating an apparent contradiction between training and validation performance.

This behaviour can be explained by the interaction between batch normalisation and the high learning rate. During training, batch normalisation uses statistics calculated from the current mini-batch, whereas during validation it relies on running mean and variance accumulated during training (Section 4.3.1). At a high learning rate, the network parameters change rapidly across training iterations, causing the running statistics to lag behind the actual activation distributions. Consequently, the inference-time normalisation becomes temporarily inaccurate, leading to unstable validation performance despite correctly learned model parameters.

The evidence in Table 5.1 supports this explanation. First, the instability occurs mainly during the high-learning-rate phase and decreases as the learning rate is reduced. Second, training loss remains stable, indicating that the issue is not caused by optimization failure. Third, validation performance fully recovers, reaching 98.39% accuracy with a validation loss of 0.0525 at epoch 19, confirming that the learned weights were effective and the earlier degradation resulted from temporary normalization mismatch.

Another contributing factor is the geometric augmentation strategy described in Section 3.5. Since augmentation is applied only to training samples, the training and validation distributions differ. Early in training, before the model learns augmentation-invariant features, this distribution gap further increases the difference between training and validation behaviour.

5.1.3 The Effect of Learning-Rate Reduction

The ReduceLROnPlateau callback halved the learning rate on five occasions, producing the schedule $10^{-3} \rightarrow 5 \times 10^{-4}$ at epoch 8, $\rightarrow 2.5 \times 10^{-4}$ at epoch 14, $\rightarrow 1.25 \times 10^{-4}$ at

epoch 23, and $\rightarrow 6.25 \times 10^{-5}$ at epoch 26.

The stabilising effect is unmistakable and can be quantified. Grouping the epochs by learning-rate regime and computing the range of validation accuracy within each group gives the summary in Table 5.2.

Table 5.2 Validation stability by learning-rate regime. As the learning rate falls, the spread of validation accuracy contracts sharply and mean validation loss decreases by more than an order of magnitude.

Learning rate	Epochs	Min val. acc.	Max val. acc.	Range	Mean val. loss
1.000×10^{-3}	1–7	48.39%	85.45%	37.06 pp	1.5382
5.000×10^{-4}	8–13	66.61%	92.69%	26.08 pp	0.7753
2.500×10^{-4}	14–22	62.80%	98.39%	35.59 pp	0.9153
1.250×10^{-4}	23–25	96.61%	98.21%	**1.60 pp**	**0.0848**
6.250×10^{-5}	26–27	97.18%	97.85%	**0.67 pp**	**0.0830**

The pattern is decisive. At the two lowest learning rates, spanning epochs 23 to 27, validation accuracy varied by at most 1.60 percentage points and mean validation loss fell to below 0.085, an eighteenfold improvement on the mean of 1.5382 observed in the initial regime. The oscillation was not merely damped but effectively eliminated. This is precisely the behaviour the mechanism described above predicts: smaller weight updates mean a slower-moving activation distribution, which the running batch normalisation estimates can track accurately, which in turn removes the train–inference mismatch responsible for the instability.

It is worth noting that the third regime, at 2.5×10^{-4} spanning epochs 14 to 22, shows a large range of 35.59 percentage points. This does not contradict the trend: this regime spans nine epochs and contains both the best validation performance of the entire run, at epoch 19, and several transient collapses at epochs 18, 20 and 21. It is a transitional regime in which the learning rate had fallen far enough to permit occasional excellent alignment but not yet far enough to make that alignment reliable. The subsequent reduction to 1.25×10^{-4} is what converted occasional excellence into consistent stability.

5.1.4 Checkpoint Selection

The oscillation raises the methodological question posed in Section 1.4: which epoch's weights should be retained when validation metrics vary by tens of percentage points between consecutive epochs? Retaining the final epoch's weights would have yielded a model with validation accuracy 97.85% and validation loss 0.0687. Retaining an arbitrary epoch could have yielded a model with validation accuracy as low as 48.39%.

The ModelCheckpoint callback resolves this principledly by tracking the minimum validation loss over the entire run and restoring those weights at termination. That minimum, 0.0525, occurred at epoch 19, where validation accuracy was 98.39%. It is these weights that were evaluated on the test partition, and this is why the test loss reported in Section 5.2 is exactly 0.0525: the same weights produce the same loss value, and the near-identical sizes of the validation and test partitions, 10,804 and 10,805 images respectively, mean the two estimates are directly comparable.

5.2 Test Set Performance

The restored best checkpoint was evaluated once on the held-out test partition of 10,805 images, which played no role in any training decision. Table 5.3 reports the result.

Table 5.3 Test set performance of the proposed lightweight CNN on the held-out partition of 10,805 images.

Evaluation metric	**Result**
Test loss	0.0525
Test accuracy	98.5007%
Correct predictions	10,643 / 10,805
Incorrect predictions	162 / 10,805
Error rate	1.4993%

A test accuracy of 98.5007% corresponds to 10,643 correct classifications out of 10,805, leaving 162 errors. Three features of this result deserve comment.

First, the correspondence between validation and test performance is close: 98.39% validation accuracy against 98.5007% test accuracy, with identical loss to four decimal places. The test accuracy is in fact marginally *higher* than the validation accuracy at the selected epoch. This is a reassuring outcome, because it indicates that the checkpoint selection procedure did not overfit to the validation set. Had the selected checkpoint been chosen because of a fortunate alignment specific to the validation data, test performance would have been noticeably worse; instead the two agree within roughly a tenth of a percentage point, which is well within the sampling variation expected for partitions of this size.

Second, the gap between final training accuracy of 99.17% and test accuracy of 98.5007% is approximately 0.67 percentage points. A gap of this magnitude is small and indicates well-controlled overfitting. The combination of geometric augmentation, dropout at rate 0.30, batch normalisation, global average pooling as a structural regulariser, and the constrained parameter budget itself has evidently been effective. A model of 248,586 parameters trained on 50,418 images has roughly five training samples per parameter, which is a comfortable ratio, and the empirical generalisation gap confirms that the capacity is appropriately matched to the data.

Third, the error rate of 1.4993% must be interpreted against the ceiling imposed by label noise discussed in Section 3.6. A proportion of the 162 errors are likely attributable to incorrectly labelled or genuinely illegible test images rather than to model failure, and the true error rate on correctly-labelled legible samples is therefore somewhat lower than 1.4993%.

5.3 Confusion Matrix Analysis

The confusion matrix is the most informative single artefact of the evaluation, because it reveals not merely how often the model errs but *in which direction*. Table 5.4 presents it in full.

Table 5.4 Confusion matrix of the proposed model on the 10,805 test images. Rows are true classes, columns are predicted classes. Diagonal entries are correct classifications. The three dominant confusions are shaded most darkly.

		Predicted Digit										
		0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**Total**
True Digit	**0**	**1073**	2	1	5	2	1	1	0	1	0	1086
	1	3	**1048**	6	1	1	0	0	0	1	19	1079
	2	0	0	**1082**	0	0	0	0	0	2	0	1084
	3	1	0	0	**1079**	0	0	1	2	2	0	1085
	4	3	3	1	1	**1079**	0	0	0	0	1	1088
	5	5	0	0	5	8	**1065**	3	0	0	0	1086
	6	0	0	0	27	0	10	**1040**	1	3	0	1081
	7	0	3	4	1	0	0	0	**1066**	1	1	1076
	8	0	0	1	0	0	0	0	0	**1072**	0	1073
	9	0	25	1	0	1	0	0	0	1	**1039**	1067
	Total	1085	1081	1096	1119	1091	1076	1045	1069	1083	1060	10805

5.3.1 Overall Structure

The matrix is strongly diagonal, as expected at 98.5007% accuracy. Its critical property, however, is that the 162 off-diagonal entries are highly *non-uniform*. Were errors distributed randomly across the 90 possible off-diagonal cells, each would contain fewer than two errors on average. Instead, a small number of cells contain values an order of magnitude larger, while the majority contain zero or one.

Table 5.5 ranks the dominant confusions.

Table 5.5 The ten most frequent confusions, ranked by count. The three leading pairs account for 71 of 162 errors, or 43.83% of all misclassifications.

Rank	Confusion	Count	% of errors	Structural interpretation
1	$6 \to 3$	27	16.67%	Loop of *chhoy* read as the lower bowl of *tin*
2	$9 \to 1$	25	15.43%	Loop of *noy* lost, leaving the dominant vertical of *ek*
3	$1 \to 9$	19	11.73%	Reverse of the above: hook of *ek* closed into a loop
4	$6 \to 5$	10	6.17%	Shared curved upper element
5	$5 \to 4$	8	4.94%	Angular lower stroke resembling the junction of *char*
6	$1 \to 2$	6	3.70%	Upper hook extended into an arc
7	$0 \to 3$	5	3.09%	Open oval resembling a bowl segment
8	$5 \to 0$	5	3.09%	Curved upper element closed into an oval
9	$5 \to 3$	5	3.09%	Double-curve ambiguity
10	$7 \to 2$	4	2.47%	Hook and oblique stroke resembling an arc
Top three combined		**71**	**43.83%**	
Top ten combined		**114**	**70.37%**	

Three confusion pairs, $6 \to 3$, $9 \to 1$ and $1 \to 9$, together account for 71 errors, or 43.83% of all misclassifications, and the ten leading confusions account for 114 errors, or 70.37%. This concentration is the central analytical finding of this thesis and directly answers RQ3.

5.3.2 Interpretation of the Dominant Confusions

The dominant confusion patterns closely match the structural similarities predicted in Table 3.1 of Chapter 3, which was based on Bangla digit geometry before evaluation. Digit 6 was predicted to be confusable with 3 and 5, and the results confirm this, with its largest errors directed toward class 3 (27 samples) and class 5 (10 samples). Similarly, digit 9 was mainly confused with 1, with 25 of its 28 errors assigned to class 1. Digit 1 showed a similar pattern, with 19 errors toward class 9 and 6 toward class 2. Thus, the observed confusion structure strongly agrees with the anticipated glyph similarities.

These results indicate that the remaining model errors are not random but are concentrated among visually similar digit pairs sharing common strokes and differing mainly in secondary features such as loops and curvature. Ambiguous handwriting, caused by rapid writing, individual style variations, or unclear stroke formation, can remove these distinguishing features and make the samples difficult even for human interpretation without contextual information.

The $1 \leftrightarrow 9$ confusion pair provides a clear example of this behaviour. The confusion is bidirectional but asymmetric, with 25 instances of 9 classified as 1 compared with 19 instances of 1 classified as 9. This occurs because losing the loop structure of 9 often leaves a shape dominated by the vertical stroke of 1, making this error more frequent than forming a loop-likestructure when writing 1.

5.3.3 Error Distribution by Class

Table 5.6 decomposes the 162 errors by class in both directions: false negatives, being errors made *on* a class, and false positives, being errors made *towards* a class.

Table 5.6 Distribution of the 162 errors by class. False negatives are row errors (samples of this class assigned elsewhere); false positives are column errors (samples of other classes assigned to this class).

Digit	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**Total**
False negatives (row)	13	31	2	6	9	21	**41**	10	1	28	162
False positives (col.)	12	33	14	**40**	12	11	5	3	11	21	162
Net (FP − FN)	−1	+2	+12	+34	+3	−10	−36	−7	+10	−7	0

The net row is revealing. Digit 3 has a strong positive net value of +34, meaning it attracts far more misassigned samples than it loses. It is an *over-attractive* class: the model's decision region for 3 is too large, absorbing samples that belong to 6, 5 and 0. Conversely digit 6 has a strong negative net value of −36, meaning it loses many samples and attracts few. Its decision region is too small, and it is being encroached upon principally by 3. The two are near-mirror images of one another, which is expected given that $6 \rightarrow 3$ is the single largest confusion in the matrix.

This asymmetry has a practical implication. The class-6/class-3 boundary is misplaced rather than merely fuzzy: the model is systematically biased in one direction along it. That is a correctable defect, and Section 6.5 discusses class-weighted or margin-based objectives that could address it.

At the other extreme, digit 8 has only 1 false negative from 1,073 test samples, and digit 2 only 2 from 1,084. These glyphs evidently possess distinctive structural features, the compound crossing of *aat* and the characteristic descending arc of *dui*, that the network detects reliably.

5.4 Classification Report and Class-Wise Analysis

Table 5.7 reports precision, recall, F1-score and support for every class, computed from the confusion matrix by Equations (4.30) to (4.32).

Table 5.7 Classification report on the test set. All values are computed from the confusion matrix of Table 5.4.

Digit	Precision	Recall	F1-Score	Support
0	0.9889	0.9880	0.9885	1086
1	0.9695	0.9713	0.9704	1079
2	0.9872	0.9982	0.9927	1084
3	0.9643	0.9945	0.9791	1085
4	0.9890	0.9917	0.9904	1088
5	0.9898	0.9807	0.9852	1086
6	0.9952	0.9621	0.9784	1081
7	0.9972	0.9907	0.9939	1076
8	0.9898	0.9991	0.9944	1073
9	0.9802	0.9738	0.9770	1067
Accuracy			**0.9850**	**10805**
Macro Avg.	0.9851	0.9850	0.9850	10805
Weighted Avg.	0.9851	0.9850	0.9850	10805

5.4.1 Aggregate Observations

Macro-averaged precision is 0.9851 and macro-averaged recall and F1-score are both 0.9850. The macro and weighted averages are identical to four decimal places, which follows directly from the near-uniform class supports produced by stratified sampling: supports range only from 1,067 to 1,088, a spread of 21 images or about 2%. This coincidence confirms that no class is over-represented and that overall accuracy is not being propped up by strong performance on an unusually large class.

The narrow range of per-class F1-scores, from 0.9704 to 0.9944, a spread of just 2.4 percentage points, is itself an important result. It demonstrates that the model's performance is genuinely balanced across all ten classes rather than concentrated in a subset of easy ones. A model achieving the same aggregate accuracy while performing near-perfectly on eight classes and poorly on two would be considerably less useful in practice, since real numeric fields contain all ten digits and the reliability of a multi-digit reading is governed by the weakest class.

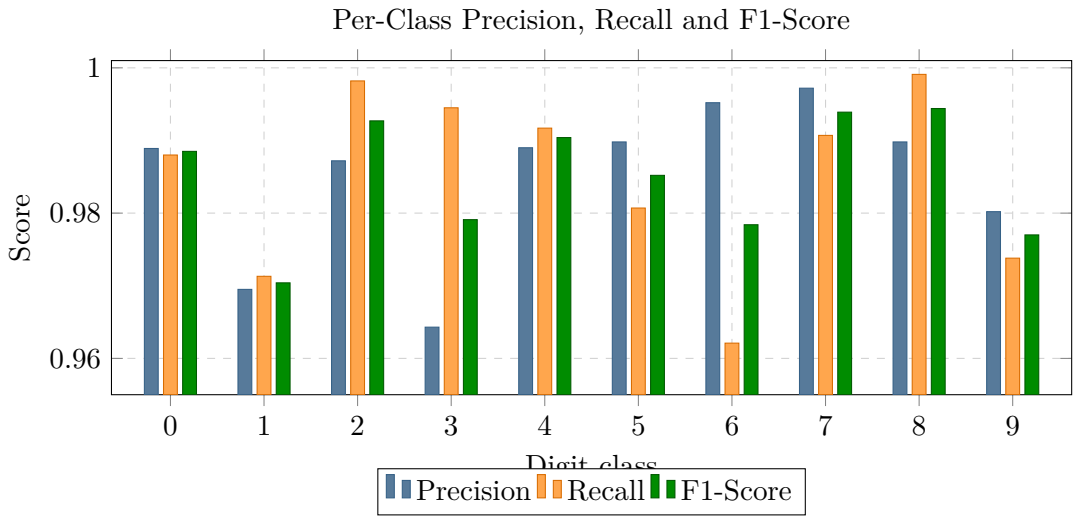

Fig. 5.3 Per-class precision, recall and F1-score. Note the divergence between precision and recall for digits 3 and 6, which reflects the asymmetric confusion between them.

5.4.2 Strongest and Weakest Classes

The strongest classes by F1-score are digit 8 at 0.9944, digit 7 at 0.9939 and digit 2 at 0.9927. Digit 8 achieves a recall of 0.9991, the highest in the report, corresponding to a single false negative in 1,073 samples. Digit 7 achieves the highest precision at 0.9972, corresponding to only three samples of other classes being misassigned to it. These glyphs are structurally distinctive and the model identifies them almost without error.

The weakest classes are digit 1 at 0.9704, digit 9 at 0.9770 and digit 6 at 0.9784. All three participate in the dominant confusion pairs of Table 5.5, and their relative weakness is therefore a direct consequence of the script-level ambiguity discussed in Section 5.3 rather than of any class-specific deficiency in training.

5.4.3 The Precision–Recall Divergence for Digits 3 and 6

The most instructive pattern in Table 5.7 is the divergence between precision and recall for digits 3 and 6, and it repays close reading.

Digit 3 has recall 0.9945, which is excellent and the second-highest in the report, but precision of only 0.9643, which is the lowest. The interpretation is unambiguous: almost every true 3 is found, but many predictions of 3 are wrong. The class is over-predicted. The confusion matrix identifies the source precisely, with 27 samples of digit 6, 5 of digit 5 and 5 of digit 0 being assigned to class 3, giving 40 false positives in total.

Digit 6 shows exactly the reverse. Its precision is 0.9952, the second-highest in the report, but its recall is 0.9621, the lowest. Almost every prediction of 6 is correct, but many true 6s are missed. Of its 1,081 test samples, 41 are assigned elsewhere, 27 of them to class 3.

Taken together these two rows describe a single misplaced decision boundary rather than two independent problems. The boundary separating the regions assigned to 3 and 6 sits too far into the territory of class 6, so that samples near the boundary are systematically resolved in favour of 3. This diagnosis is far more actionable than the aggregate F1-scores of 0.9791 and 0.9784 would suggest in isolation, and it exemplifies why the confusion matrix was treated in this thesis as a primary object of analysis rather than as a summary artefact.

5.5 Real Handwritten Image Prediction

To assess behaviour beyond the curated corpus, and thereby address RQ5, the trained model was applied to photographed handwritten digits captured outside the acquisition conditions of NumtaDB. These samples were written with an ordinary pen on ordinary paper and photographed with a mobile device under uncontrolled ambient lighting.

Because such images differ substantially from the corpus in resolution, illumination, contrast and framing, a dedicated inference-time preprocessing pipeline was required, comprising five stages: conversion of the photograph to grayscale; binarisation by thresholding to separate ink from a non-uniformly illuminated background, for which a global histogram-based threshold of the kind introduced by Otsu [24] is the standard approach; extraction and centring of the digit within its bounding region; resizing to the 32×32 input resolution; and normalisation to the unit interval. This pipeline mirrors that of Section 3.4 with the addition of the thresholding and centring stages made necessary by photographic capture.

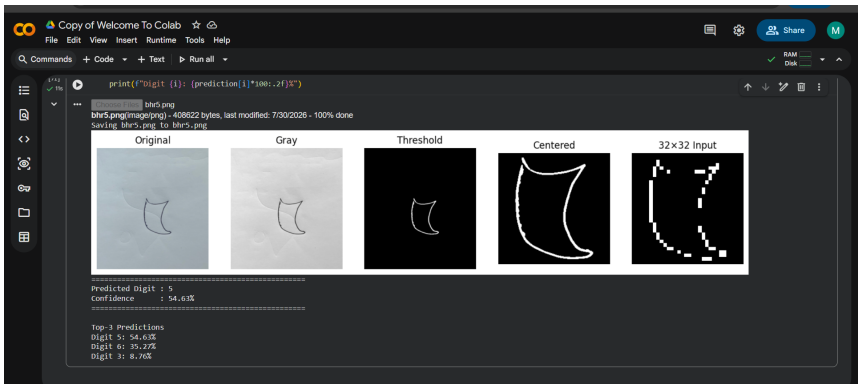

Fig. 5.4 Out-of-distribution prediction on a photographed handwritten digit.

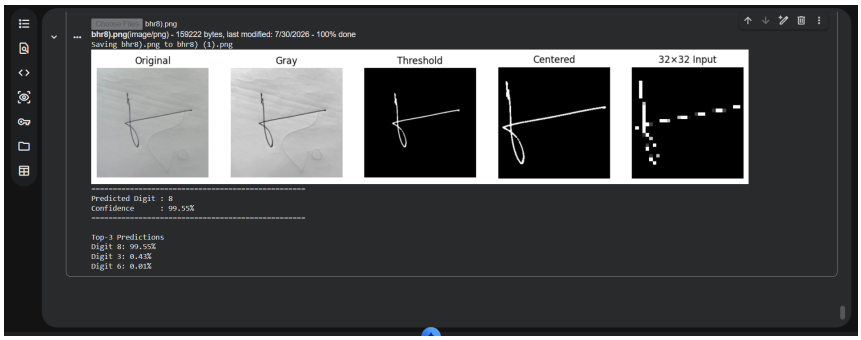

Fig. 5.5 Second out-of-distribution prediction on a photographed handwritten digit.

The two cases illustrate opposite ends of the confidence spectrum and are informative precisely because of that contrast.

In the second case the model predicted digit 8 with 99.55% confidence, with the runner-up class 3 receiving only 0.43% and the third-placed class 6 only 0.01%. This is a decisive, well-calibrated prediction: the sample was clearly written, the distinctive compound structure of *aat* survived the photographic pipeline intact, and the model recognised it without hesitation. This is consistent with the corpus result that digit 8 is the strongest class by recall.

In the first case the model predicted digit 5 with only 54.63% confidence, with digit 6 a close second at 35.27% and digit 3 third at 8.76%. The prediction is nominally correct

but the margin is narrow. Two observations follow. First, the confidence distribution is honest: the model expressed genuine uncertainty rather than assigning spurious high confidence to a difficult sample, which is desirable behaviour and suggests the softmax outputs carry usable calibration information. Second, and more tellingly, the competing classes are 6 and 3, which together with 5 form exactly the confusion cluster identified in Table 5.5: the corpus results record 10 instances of $6 \rightarrow 5$, 5 of $5 \rightarrow 3$ and 27 of $6 \rightarrow 3$. The same structural ambiguity that dominates the in-corpus confusion matrix reappears, unprompted, in an out-of-distribution photograph.

This convergence is a meaningful validation of the error analysis. The confusion structure identified in Section 5.3 is not an artefact of the NumtaDB test partition; it reflects a property of the Bangla numeral system that persists under entirely different acquisition conditions.

The wider conclusion is one of measured caution. Performance on photographed input is evidently degraded relative to the 98.5007% obtained on curated test images, since a 54.63% confidence on a clearly-written sample would be unusual within the corpus. The model has learned the NumtaDB distribution well, and photographic capture constitutes a genuine distribution shift that the geometric augmentation of Section 3.5, which models rotation, translation and scale but not illumination, contrast or blur, does not fully cover. This is a limitation, recorded as such in Section 6.4, and it directly motivates the photometric augmentation proposed in Section 6.5. It should be emphasised that this assessment is qualitative and based on a small number of samples; it is indicative rather than statistically conclusive.

5.6 Comparison with Previous Studies

Table 5.8 positions the proposed model against representative prior work on Bangla handwritten digit recognition.

Table 5.8 Comparison with representative prior studies. Accuracies from other work are reported under differing partition protocols and differing treatment of duplicates and augmentation, and are therefore *not* directly comparable; they indicate the general performance regime only.

Study / approach	**Dataset**	**Method**	**Accuracy**	**Model size**
Classical pipeline [11]	CMATERdb	GA feature selection + classifier	∼97%†	Not reported
CNN + autoencoder [31]	CMATERdb / ISI	Pretraining + CNN	∼99%†	Not reported
Deep CNN backbone	NumtaDB	ResNet / DenseNet family	>99%†	Millions of params
Ensemble CNN	NumtaDB	Multi-model ensemble	>99%†	Multiples of above
Proposed	**NumtaDB (deduplicated)**	**Lightweight DSC-CNN, residual, GAP**	**98.5007%**	**248,586 params (≈1 MB)**

† Not directly comparable. Differences in dataset version, partition protocol, duplicate handling and augmentation prevent controlled comparison. Reproduced to indicate the performance regime only.

Two caveats must be stated plainly before drawing any conclusion from this table.

First, the comparison is not controlled. Prior studies use different dataset versions, different train–test protocols, different augmentation regimes and, critically, different or unstated treatment of duplicate images. A study that partitions NumtaDB without deduplication is measuring something subtly different from what is measured here, as Section 3.3 explained. A difference of a few tenths of a percentage point between studies conducted under different protocols carries little information.

Second, and more importantly, the comparison that matters for this thesis is not accuracy alone but the accuracy-per-parameter trade-off. The proposed model attains 98.5007% with 248,586 parameters. Deep backbone models reported on NumtaDB typically contain parameters numbering in the millions to tens of millions. If such a model attains, say, 99.3% accuracy with ten million parameters, it purchases roughly 0.8 additional percentage points of accuracy at a cost of roughly forty times the parameter count. Whether that trade is worthwhile depends entirely on the deployment context. For a server-side batch processing system with abundant compute it plainly is. For an offline mobile application running on a low-cost handset in a rural bank branch, it plainly is not, and a model that cannot be deployed delivers zero accuracy in practice regardless of its benchmark score.

The contribution claimed here is therefore not that the proposed model is the most accurate Bangla digit classifier, which it is not, but that it occupies a markedly more favourable position on the accuracy–efficiency frontier than the models that dominate the literature, and that this position is the relevant one for the deployment settings identified in Section 1.2.

5.7 Discussion

The results support several important conclusions.

The capacity constraint was not the limiting factor. The proposed model achieved 98.5007% test accuracy with only 248,586 parameters, demonstrating that a lightweight architecture can achieve strong performance without large model capacity. The training accuracy of 99.17% indicates that the model retained sufficient capacity, and the main challenge was generalisation rather than underfitting. This finding answers RQ1 and suggests that larger architectures commonly used for this task may be unnecessarily complex.

Efficiency was achieved through architectural choices. Depthwise separable convolution reduced the feature extractor parameters by approximately 8.6×, from 387,360 to 45,065 weights, while global average pooling eliminated the need for a large dense layer containing 524,288 parameters. These two design decisions were the primary contributors to model compactness and directly address RQ2.

Residual errors reflect script-level ambiguity. Three major confusion pairs accounted for 43.83% of all classification errors, and these pairs were predicted from Bangla digit geometry before evaluation. This demonstrates that remaining errors are mainly caused by visually similar glyph structures rather than random model failures. Therefore, improving specific decision boundaries may be more effective than increasing model size, addressing RQ3.

Training strategy was essential for reliable performance. The initial training phase showed severe validation instability, with fluctuations exceeding thirty percentage points and a maximum validation loss of 3.4994. The combination of ReduceLROnPlateau and ModelCheckpoint reduced this instability and preserved the best-performing epoch-19 model with a validation loss of 0.0525. This confirms the importance of training configuration in achieving stable results and answers RQ4.

Generalisation to unseen conditions remains limited. The real-image evaluation demonstrated both successful recognition and uncertain predictions under photometric variations. The marginal cases followed the same confusion patterns observed in the dataset, indicating that the model learned the NumtaDB distribution effectively but lacks robustness to unseen image conditions. Photometric augmentation is therefore a potential future improvement, addressing RQ5.

5.8 Summary

This chapter presented the complete experimental evaluation of the proposed lightweight CNN model. The network achieved 99.17% training accuracy and 98.5007% test accuracy on 10,805 held-out images, producing a test loss of 0.0525 with 10,643 correct predictions. Macro-averaged precision, recall, and F1-score were approximately 0.985, while individual class F1-scores ranged from 0.9704 to 0.9944. The analysis explained early validation instability through the interaction between batch normalisation and large weight updates, and showed how learning-rate scheduling improved stability. Confusion analysis revealed that most errors originated from a small number of Bangla glyph similarities rather than general model weakness. The results demonstrate that the proposed lightweight architecture provides a strong accuracy-efficiency balance and is suitable for resource-constrained handwritten digit recognition applications.

Chapter 6

Conclusion and Future Work

6.1 Summary of the Research

This thesis investigated whether Bangla handwritten digit recognition can be performed effectively using a lightweight CNN suitable for deployment on low-cost mobile and embedded devices. The motivation was to achieve a balance between recognition accuracy and computational efficiency, as highly accurate but resource-intensive models are often unsuitable for practical applications.

A lightweight CNN architecture was developed using depthwise separable convolutions to reduce computational cost while maintaining effective feature learning. The model consists of four convolutional blocks with channel sizes of 32, 64, 128, and 256, each incorporating batch normalisation, ReLU activation, max pooling, and residual connections. Instead of conventional flattening, global average pooling was employed to reduce the feature representation size, followed by a 128-unit dense layer with dropout and a ten-class softmax classifier. The final model contains only 248,586 parameters,with a storage requirement of approximately one megabyte.

The model was trained and evaluated on NumtaDB after preprocessing images into grayscale 32×32 representations and normalising pixel values. Duplicate removal was performed before data partitioning, resulting in 72,027 unique samples divided into training, validation, and test sets. Training used the Adam optimiser, sparse categorical cross-entropy loss, batch size of 64, geometric augmentation, and callback strategies including early stopping, learning-rate reduction, and checkpoint restoration. Training stopped after 27 epochs while preserving the best-performing model.

6.2 Major Findings

Five major findings were obtained corresponding to the five research questions presented in Section 1.3.1.

A lightweight architecture can achieve competitive accuracy. The proposed model achieved 98.5007% test accuracy with a test loss of 0.0525 on 10,805 test images, correctly classifying 10,643 samples. The training accuracy of 99.17% indicates that the model had sufficient capacity, while the small gap between training and test performance demonstrates controlled overfitting. Therefore, RQ1 is answered affirmatively: high recognition performance can be achieved without large-scale architectures.

Efficiency was mainly achieved through two architectural choices. Depthwise separable convolution reduced the feature extractor parameters from 387,360 to 45,065 weights, providing approximately an 8.6× reduction compared with standard convolution. In addition, global average pooling eliminated the need for a 524,288-parameter dense layer

required by flattening. Together, these choices produced the compact architecture and quantitatively answer RQ2.

The remaining errors are related to Bangla glyph similarity. The 162 misclassifications were concentrated among a small number of digit pairs. The three dominant confusions (6→3, 9→1, and 1→9) contributed 71 errors, representing 43.83% of all failures. These patterns matched the structural similarities predicted from Bangla digit geometry before testing. The results indicate that errors mainly arise from inherent script ambiguity rather than random model weakness, answering RQ3.

Training strategy was essential for stable performance. During the high-learning-rate phase, validation accuracy fluctuated between 48.39% and 85.45%, with validation loss reaching 3.4994, despite stable training improvement. This behaviour was attributed to the mismatch between batch statistics used during training and running statistics used during inference in batch normalisation. Learning-rate reduction reduced validation variation from 37.06 to 0.67 percentage points, while checkpoint restoration preserved the best epoch-19 model with validation loss 0.0525. This confirms that the training configuration was a critical component of the method, answering RQ4.

Generalisation to unseen conditions remains limited. Evaluation on photographed handwriting showed both strong and uncertain cases. One sample was classified as digit 8 with 99.55% confidence, while another produced a marginal prediction of digit 5 with 54.63% confidence. The uncertain case followed the same confusion pattern observed in the dataset, showing that the model captures script-level ambiguity but remains sensitive to image variations not represented during training. Thus, RQ5 is answered with the conclusion that additional photometric augmentation could improve robustness.

6.3 Contribution

The contributions of this thesis, as set out in Section 1.6 and substantiated in the chapters that followed, are as follows. It demonstrates that Bangla handwritten digit recognition accuracy of 98.5007% is attainable at a parameter budget of 248,586, roughly an order of magnitude below that of the deep architectures most frequently reported for this task, thereby occupying a markedly more favourable position on the accuracy–efficiency frontier than the prevailing literature. It provides a formal, architecture-specific efficiency analysis rather than a generic appeal to published efficiency claims, quantifying the saving block by block. It documents a fully reproducible preparation protocol including explicit duplicate removal before partitioning, a step whose omission can silently inflate reported accuracy on assembled corpora and which is rarely reported in this literature. It supplies a detailed error analysis that links the model's residual mistakes to identifiable structural properties of the Bangla numeral system, converting an opaque accuracy figure into actionable diagnostic information. And it documents and explains a training instability that is widely encountered but seldom analysed, showing how a principled callback regime converts it into a reliable outcome.

6.4 Limitations

The limitations of this work should be acknowledged to properly interpret the reported results.

No ablation study. The proposed architecture combines depthwise separable convolution, residual connections, batch normalisation, global average pooling, and dropout. Although the parameter reduction from separable convolution and global average pooling was analytically quantified, the individual accuracy contribution of each component was not experimentally evaluated through ablation studies.

Single training run. All results were obtained from a single training run with one random seed. Since neural network optimisation is stochastic, small variations in accuracy may occur across different runs. Therefore, the reported 98.5007% accuracy should be considered a point estimate rather than an exact reproducible value.

Limited duplicate detection. The deduplication process identified only exact pixel-level duplicates, removing 18 images. Near-duplicate samples caused by transformations, intensity changes, or noise may still exist, particularly because NumtaDB contains augmented data [1]. Therefore, a small amount of residual evaluation bias may remain.

No hardware-based efficiency evaluation. Efficiency was evaluated using parameter counts and theoretical computational cost rather than real hardware measurements. Actual inference speed, memory usage, and energy consumption depend on hardware architecture and software implementation, and therefore the practical speed improvement was not directly verified.

No model compression. The study focused only on architectural efficiency. Techniques such as quantisation, pruning, and knowledge distillation were not applied. Therefore, the reported approximately one-megabyte model size represents the uncompressed single-precision version.

Limited out-of-distribution evaluation. The real-image evaluation was performed on a small number of photographed samples and provides qualitative evidence only. A larger evaluation covering different lighting conditions, writing instruments, paper types, and camera devices is required to measure robustness under distribution shift.

Recognition limited to isolated digits. The study focuses on pre-segmented Bangla handwritten digits. Recognition from complete document images, multi-digit number strings, and complex Bangla characters remains outside the current scope. A complete handwriting recognition system would require additional detection and segmentation components, whose errors may further affect recognition performance.

6.5 Future Directions

The findings and limitations of this study suggest several directions for future research, prioritised according to expected benefit and implementation effort.

Photometric augmentation. The real-image evaluation showed that the current geometric augmentation strategy does not sufficiently address variations caused by illumination, contrast, blur, and sensor noise. Adding brightness variation, gamma adjustment, Gaussian blur, noise injection, and simulated lighting conditions could improve robustness to photographic inputs without increasing model size or inference cost.

Confusion-aware training strategies. Since 43.83% of errors originate from three major confusion pairs, targeted learning strategies may provide greater improvement than increasing model capacity. Possible approaches include class-weighted losses, margin-based objectives, focal loss, and hierarchical classifiers designed specifically for confusable Bangla digit groups.

Quantisation and pruning. Applying 8-bit quantisation could significantly reduce the model size from approximately one megabyte to around 249 kilobytes. Structured pruning, particularly in high-parameter layers, could further improve efficiency. However, the accuracy impact of these compression methods should be carefully evaluated.

Ablation and multi-seed evaluation. Future studies should perform systematic ablation by removing or replacing individual components such as depthwise convolution, residual connections, global average pooling, and dropout. Repeated experiments with multiple random seeds would also provide more reliable performance estimates through confidence intervals.

On-device benchmarking. Actual deployment evaluation on mobile and embedded hardware is required to validate practical efficiency. Measurements of inference latency, memory usage, and energy consumption would provide stronger evidence than analytical operation counts alone.

Improved duplicate detection. Replacing exact-match deduplication with perceptual hashing or feature-space similarity methods could identify near-duplicate samples and provide a more accurate estimate of model generalisation.

Extension to Bangla character recognition. The proposed architecture can be extended beyond digits to Bangla basic and compound characters. Evaluating its scalability on a larger label space would determine whether the lightweight design remains effective for more complex recognition tasks.

Complete handwriting recognition pipeline. A future system could integrate digit detection, segmentation, and sequence recognition models to process complete document images and multi-digit fields. Such an end-to-end evaluation would better represent real-world applications.

6.6 Concluding Remarks

The central message of this thesis is that model size and model usefulness are not the same thing, and that in low-resource settings they can be in direct opposition. The prevailing incentive in the literature rewards incremental accuracy on benchmark partitions, and that

incentive has produced Bangla digit recognisers of impressive accuracy and impractical size. This thesis has shown that a model of 248,586 parameters, small enough to ship inside a mobile application without special provision, reaches 98.5007% test accuracy on NumtaDB, and that the accuracy it forgoes relative to far larger models is modest while the resources it forgoes are not.

It has also shown that the errors such a model makes are not random noise to be reduced by brute force but a structured, interpretable signal. The three confusion pairs responsible for nearly half of all misclassifications were predictable in advance from the geometry of the Bangla script, and they reappeared unprompted in photographed handwriting captured under entirely different conditions. That regularity is more valuable than a marginally better accuracy figure would have been, because it tells the next researcher where to look.

Finally, the training instability documented here, with validation loss peaking at 3.4994 while training loss fell smoothly below 0.11, is a reminder that apparently alarming behaviour in a learning curve may have a mundane explanation and a principled remedy. Understanding the mechanism, in this case the lag of batch normalisation running statistics behind rapidly moving weights, turned a worrying training run into a reliable result. Diagnosis rather than escalation is often the more productive response, and that methodological lesson generalises well beyond the recognition of Bangla numerals.

References

[1] Samiul Alam, Tahsin Reasat, Rashed Mohammad Doha, and Ahmed Imtiaz Humayun. NumtaDB – assembled bengali handwritten digits. *arXiv preprint arXiv:1806.02452*, 2018.

[2] Md Zahangir Alom, Paheding Sidike, Tarek M. Taha, and Vijayan K. Asari. Handwritten bangla digit recognition using deep learning. *arXiv preprint arXiv:1705.02680*, 2017.

[3] Md Zahangir Alom, Paheding Sidike, Mahmudul Hasan, Tarek M. Taha, and Vijayan K. Asari. Handwritten bangla character recognition using the state-of-the-art deep convolutional neural networks. *Computational Intelligence and Neuroscience*, 2018: 6747098, 2018.

[4] Ujjwal Bhattacharya and Bidyut B. Chaudhuri. Handwritten numeral databases of indian scripts and multistage recognition of mixed numerals. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(3):444–457, 2009.

[5] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.

[6] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[7] François Chollet. Xception: Deep learning with depthwise separable convolutions. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1251–1258. IEEE, 2017.

[8] Dan Ciresan, Ueli Meier, and Jürgen Schmidhuber. Multi-column deep neural networks for image classification. In *2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3642–3649. IEEE, 2012.

[9] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20 (3):273–297, 1995.

[10] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 886–893. IEEE, 2005.

[11] Nibaran Das, Ram Sarkar, Subhadip Basu, Mahantapas Kundu, Mita Nasipuri, and Dipak Kumar Basu. A genetic algorithm based region sampling for selection of local features in handwritten digit recognition application. *Applied Soft Computing*, 12(5): 1592–1606, 2012.

[12] Li Deng. The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.

[13] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.

[14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778. IEEE, 2016.

[15] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. Searching for MobileNetV3. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1314–1324. IEEE, 2019.

[16] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[17] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141. IEEE, 2018.

[18] Ming-Kuei Hu. Visual pattern recognition by moment invariants. *IRE Transactions on Information Theory*, 8(2):179–187, 1962.

[19] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4700–4708. IEEE, 2017.

[20] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, volume 25, pages 1097–1105, 2012.

[21] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[22] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553): 436–444, 2015.

[23] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.

[24] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979.

[25] Umapada Pal and Bidyut B. Chaudhuri. Indian script character recognition: a survey. *Pattern Recognition*, 37(9):1887–1899, 2004.

[26] Réjean Plamondon and Sargur N. Srihari. On-line and off-line handwriting recognition: a comprehensive survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(1):63–84, 2000.

[27] A. K. M. Shahariar Azad Rabby, Sadeka Haque, Md. Sanzidul Islam, Sheikh Abujar, and Syed Akhter Hossain. BornoNet: Bangla handwritten characters recognition using convolutional neural network. *Procedia Computer Science*, 143:528–535, 2018.

[28] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520. IEEE, 2018.

[29] S. M. A. Sharif, Nabeel Mohammed, Nafees Mansoor, and Sifat Momen. A hybrid deep model with HOG features for bangla handwritten numeral classification. In *2016 9th International Conference on Electrical and Computer Engineering (ICECE)*, pages 463–466. IEEE, 2016.

[30] Ashadullah Shawon, Md. Jamil Ur Rahman, Firoz Mahmud, and M. M. Arefin Zaman. Bangla handwritten digit recognition using deep CNN for large and unbiased dataset. In *2018 International Conference on Bangla Speech and Language Processing (ICBSLP)*. IEEE, 2018.

[31] Md. Shopon, Nabeel Mohammed, and Md. Anowarul Abedin. Bangla handwritten digit recognition using autoencoder and deep convolutional neural network. In *2016 International Workshop on Computational Intelligence (IWCI)*, pages 64–68. IEEE, 2016.

[32] Laurent Sifre and Stéphane Mallat. Rigid-motion scattering for texture classification. *arXiv preprint arXiv:1403.1687*, 2014.

[33] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

[34] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.

[35] Abu Sufian, Anirudha Ghosh, Avijit Naskar, Farhana Sultana, Jaya Sil, and M. M. Hafizur Rahman. BDNet: Bengali handwritten numeral digit recognition based on densely connected convolutional neural networks. *Journal of King Saud University – Computer and Information Sciences*, 2020.

[36] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9. IEEE, 2015.

[37] Andreas Veit, Michael J. Wilber, and Serge Belongie. Residual networks behave like ensembles of relatively shallow networks. In *Advances in Neural Information Processing Systems (NIPS)*, volume 29, pages 550–558, 2016.

[38] Hasib Zunair, Nabeel Mohammed, and Sifat Momen. Unconventional wisdom: A new transfer learning approach applied to bengali numeral classification. In *2018 International Conference on Bangla Speech and Language Processing (ICBSLP)*. IEEE, 2018.